排序、選擇與隨機分析

排序要決定所有資料的先後；選擇只問其中一個秩。兩者可以共用分割與堆積，卻不必付出相同成本。

本章依序學：**比較模型的限制 → 堆積與排序 → 隨機分析工具 → 分割與快速排序 → 計數、基數與桶排序 → 次序統計量**。最後用生日問題、負載、等待與線上遴選練習機率建模。

先備知識：第 1 章的規格、不變量、級數，第 2 章的分治與遞迴式。正文先補齊本章需要的機率工具，再使用期望成本。

參考用書：Cormen 等，*Introduction to Algorithms*, 3rd Edition，Chapters 5–9；補充參考 MIT 6.046J（2005）Lectures 4–6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先定義排序的對象與保證

每筆資料由 **key（鍵值）** 與隨它搬動的欄位組成。另加身分標籤 a, b, c，方便辨識相同鍵的不同資料；除非特別說明，比較只看 key。

正確排序必須同時保證：輸出鍵值非遞減，而且輸出是輸入資料的同一個多重集合。**穩定性**另要求同鍵資料維持原先順序。

例輸入 $[2_a, 1_b, 2_c]$：$[1_b, 2_a, 2_c]$ 穩定；$[1_b, 2_c, 2_a]$ 排序正確但不穩定；$[1_b, 2_a, 2_a]$ 遺失 2_c，連排序規格都不符合。

本章陣列採 **1 起算、閉區間** $A[p..r]$，$p > r$ 表示空區間；鍵值計數表則依鍵值從 0 起算。選擇的 k 永遠是目前子區間內、1 起算的局部秩。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較數、執行時間與空間各自計算

預設比較與搬移一筆固定大小資料、讀寫一個索引均為常數成本。若鍵是長字串、大整數或外部儲存物件，必須把實際比較、取位與資料搬移成本加回來。

要回答的問題	必須固定的條件
最壞時間 $\max_A T(A)$	長度與合法輸入集合
平均時間 $\mathbb{E}_A[T(A)]$	輸入的聯合分布
隨機演算法 $\mathbb{E}_R[T(A,R)]$	固定輸入 A，對內部亂數 R 平均
額外空間	輸出陣列、暫存區及同時存活的呼叫框架是否計入

「分割原地完成」只說分割程序的儲存需求；快速排序仍可能累積遞迴堆疊。「期望很快」也沒有排除某條亂數路徑很慢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較決策樹：每片葉要辨識一種次序

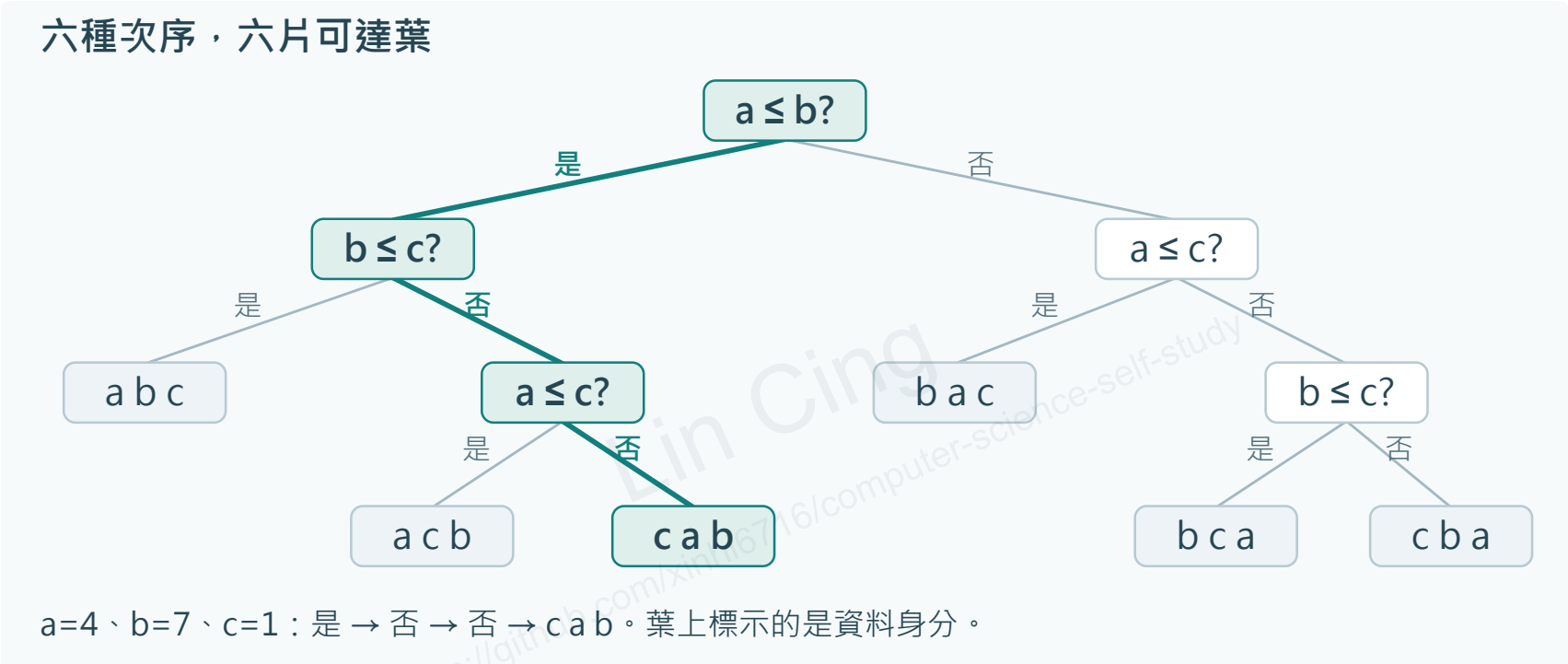
先限制 n 個鍵互異。比較排序只能由兩鍵比較取得次序資訊，不能讀鍵值當作陣列索引；固定輸入長度後，用二元樹表示所有執行。

內節點標示被比較的兩個原始資料身分；左右邊分別表示 $\leq$、$>$ 的結果。根到葉記錄本次觀察，葉必須決定這 n 個身分的完整排序。

$n!$ 種相對次序必須導向不同的可達葉。若兩種次序到同一葉，演算法觀察完全相同，只能輸出同一個身分排列，至少有一種會錯。因此可達葉數至少 $n!$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

沿著比較結果，走到資料的次序



圖中的葉標示的是身分次序；例如 c, a, b 表示 $c < a < b$。取 $a = 4, b = 7, c = 1$，依比較結果追蹤到 c, a, b，即可核對一條執行路徑。

最壞下界說的是「至少有一個輸入」

高度 h 的二元樹最多有 2^h 片葉，所以

$$n! \le 2^h, \qquad h \ge \lceil \lg(n!) \rceil.$$

對 $n \ge 2$，階乘的後半至少有 $\lfloor n/2 \rfloor$ 個因子，各不小於 $n/2$；另一方面每個因子至多 n。因此

$$\lfloor n/2 \rfloor \lg(n/2) \le \lg(n!) \le n \lg n.$$

對充分大 n，兩側給出 $\lg(n!) = \Theta(n \lg n)$；小規模另以常數涵蓋。

結論：**每個正確的比較排序，都有最壞輸入需要 $\Omega(n \lg n)$ 次比較。** 這沒有宣稱每個輸入都慢；插入排序遇已排序輸入只做 $n - 1$ 次鍵比較。

允許重複鍵仍包含互異鍵這個困難子集。計數與基數排序則利用鍵值表示，已離開比較模型，必須另外計鍵範圍與位元數的成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

平均下界不能只看最深的葉

令 $N = n!$ 種相對次序均勻，各選一片代表葉，深度為 $d_1, \dots, d_N$。不同葉的根路徑彼此不是前綴。把每個深度 d_i 的葉對應到 $[0, 1)$ 中長度 2^{-d_i} 的二元區間，這些區間互不相交，故

$$\sum_{i=1}^{N} 2^{-d_i} \le 1.$$

由幾何平均不大於算術平均，

$$2^{-\frac{1}{N}\sum_i d_i} \le \frac{1}{N}\sum_i 2^{-d_i} \le \frac{1}{N}.$$

取對數得到 $\frac{1}{N}\sum_i d_i \ge \lg N = \Omega(n \lg n)$。這才是均勻輸入的平均下界；不能由最深路徑直接推到平均路徑。

對每條亂數路徑皆正確的排序，固定亂數後仍有此下界。再對亂數平均，因最大值不小於均勻輸入平均，可得 $\max_A E_r[C(A, r)] \ge \lg(n!)$；允許錯答的排序不在此論證範圍。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

三鍵數例：區分輸入平均與亂數平均

用三鍵數例區分兩種平均。L 採前述的三鍵決策樹；R 採相同排序程序，但將原始身分順序改為 a, c, b。排序前等機率選 L 或 R：

鍵值遞增的身分次序	L 比較數	R 比較數	兩者平均
a, b, c	2	3	$5/2$
a, c, b	3	2	$5/2$
b, a, c	2	3	$5/2$
b, c, a	3	3	3
c, a, b	3	2	$5/2$
c, b, a	3	3	3

各欄對均勻輸入平均皆為 $16/6 = 8/3 \geq \lg 6$，各列期望的最大值為三。均勻輸入平均、單一輸入對亂數的平均、最壞期望成本，三者不可混用。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

堆積：形狀限制與鍵值限制

二元堆積的形狀是**完全二元樹**：最後一層以前全滿，最後一層由左往右填。1 起算陣列的位置 i 有

$$parent(i) = \lfloor i/2 \rfloor, \quad left(i) = 2i, \quad right(i) = 2i + 1.$$

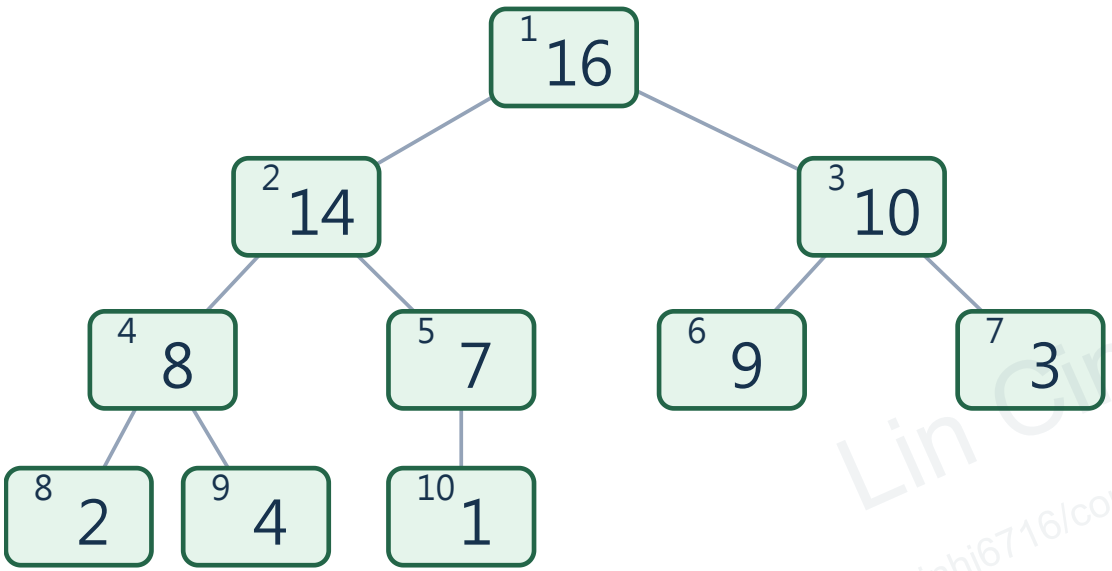
只有 $1 \le i \le s$ 的位置屬於目前堆積，s 是 heap-size（有效堆積大小）。陣列容量、陣列長度與 s 是不同的量。

max-heap 要求每個非根位置 i 都有 $A[parent(i)] \ge A[i]$；此處 $A[i]$ 簡寫其鍵值，交換時仍搬動整筆資料。沿祖先鏈可知根為全體最大鍵；同層或左右子樹之間沒有完整排序關係。

高度計最長向下路徑的**邊數**；單節點高度 0，非空 n 節點堆積高 $\lfloor \lg n \rfloor$。若改計層數，則是 $\lfloor \lg n \rfloor + 1 = \lceil \lg(n+1) \rceil$。空堆積另行處理，不代入 $\lg 0$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一棵堆積的樹與陣列表示



節點左上：索引　中央：鍵值

索引固定，鍵值移動

1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

每條邊都滿足：父鍵 ≥ 子鍵

同一列的節點不保證有序

節點旁標示陣列索引。交換時鍵值移動，位置的父子索引關係不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

向下修復的前提不能省略

前提：$1 \le i \le s$，位置 i 的左右子樹已各自為 max-heap，只有 i 與子節點之間可能違規。不存在的子節點不讀取；and 採短路求值。

```
MAX-HEAPIFY(A,i,s):
    while true:
        largest = i
        if 2*i <= s and A[2*i] > A[largest]:
            largest = 2*i
        if 2*i+1 <= s and A[2*i+1] > A[largest]:
            largest = 2*i+1
        if largest == i: return
        swap(A[i],A[largest])
        i = largest
```

每輪在目前節點與存在的兩個子節點中找最大者，再決定是否交換。只比較兩個子節點、沒有比較父節點，並不能判斷是否需要修復。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

向下修復為何正確，又為何會停止

若最大者就是目前節點，兩條父子邊合法；兩棵子樹原本合法，因此整棵子樹完成。

否則把較大子節點的根鍵提升。它不小於自己的後代，也不小於另一個子節點的根與原父鍵，所以提升後父節點的兩條邊都合法。唯一可能的新違規，在舊父鍵下移的位置；另一棵子樹保持不變。

對剩餘高度歸納即可證明整棵子樹最後合法。每次 i 變為 $2i$ 或 $2i+1$，嚴格下降一層；高度 h 至多下降 h 次，所以時間 $O(h+1)$，迭代版額外空間 $O(1)$。

例 $A=[16,4,10,8,7,9,3]$、$i=2, s=7$：選子鍵 8，交換位置 $2,4$，得 $[16,8,10,4,7,9,3]$。位置 4 的子索引 $8,9$ 已超界，停止。索引 7 的鍵 3 屬於另一棵子樹，這一輪不會拿它比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

由下而上建堆：先讓子樹符合前提

```
BUILD-MAX-HEAP(A):
    s = length(A)
    for i = floor(s/2) downto 1:
        MAX-HEAPIFY(A,i,s)
```

不變量：每輪開始時，索引大於 i 的每個節點都是合法 max-heap 的根。

初始化 $i = \lfloor n/2 \rfloor$ 時，較大索引全是葉，成立。兩個子節點索引都比 i 大，因此呼叫滿足 MAX-HEAPIFY 的前提；完成後 i 也合法，而且其後代仍是合法子堆。令 i 減一，保持下一輪條件。

終止 $i = 0$ 時，根 1 也合法，整棵堆積完成。$n = 0, 1$ 時不執行迴圈，直接得到合法空堆或單節點堆。

$\lfloor n/2 \rfloor$ 是最後一個可能有子節點的位置，**不是第一片葉**；例如 $n = 10$ 時位置 5 還有左子節點 10，不能漏掉這一輪。

由位置五往前建堆：每輪完整狀態

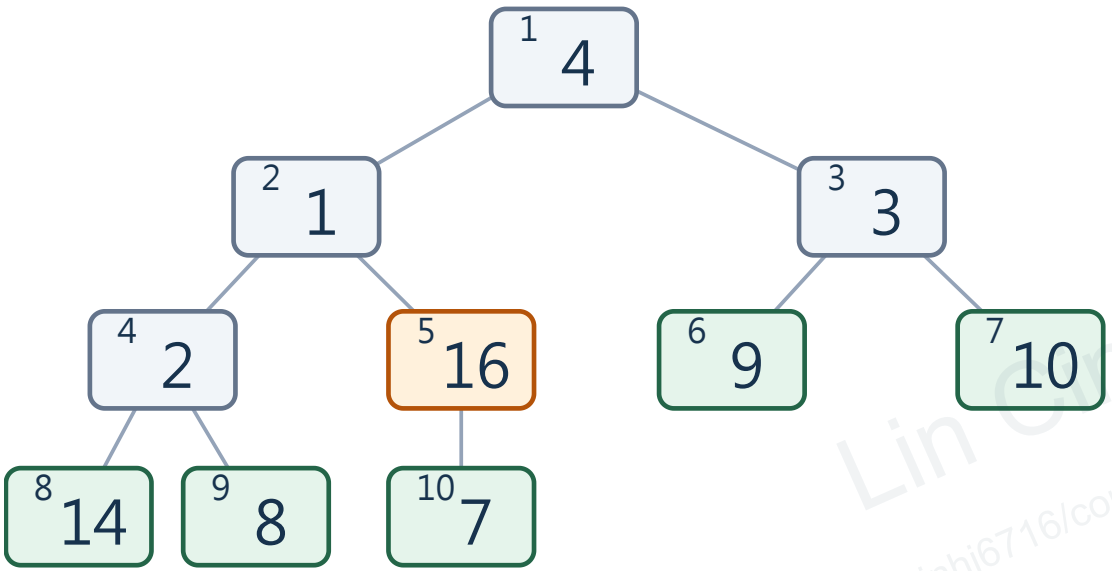
輸入 $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$，有效大小 $s = 10$。表格採 **1 起算索引**；每輪的兩棵子樹已先修復，才處理本輪的根。

根 i	選子節點與交換索引	本輪修復後 $A[1..10]$
5	唯一子鍵 7；$16 \geq 7$，不交換	$[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$
4	子鍵 $14, 8$，選 14；$4 \leftrightarrow 8$	$[4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$
3	子鍵 $9, 10$，選 10；$3 \leftrightarrow 7$	$[4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$
2	選 16，再選 7；$2 \leftrightarrow 5$、$5 \leftrightarrow 10$	$[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$
1	選 $16, 14, 8$；$1 \leftrightarrow 2$、$2 \leftrightarrow 4$、$4 \leftrightarrow 9$	$[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

接下來的樹與陣列圖使用同一組資料。橘框為目前處理的根，綠框為已保證合法的子樹根；位置的父子關係固定，交換只移動鍵值及其資料。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

位置五：只有左子節點



節點左上：索引　中央：鍵值

索引固定，鍵值移動

1	2	3	4	5
4	1	3	2	16
6	7	8	9	10
9	10	14	8	7

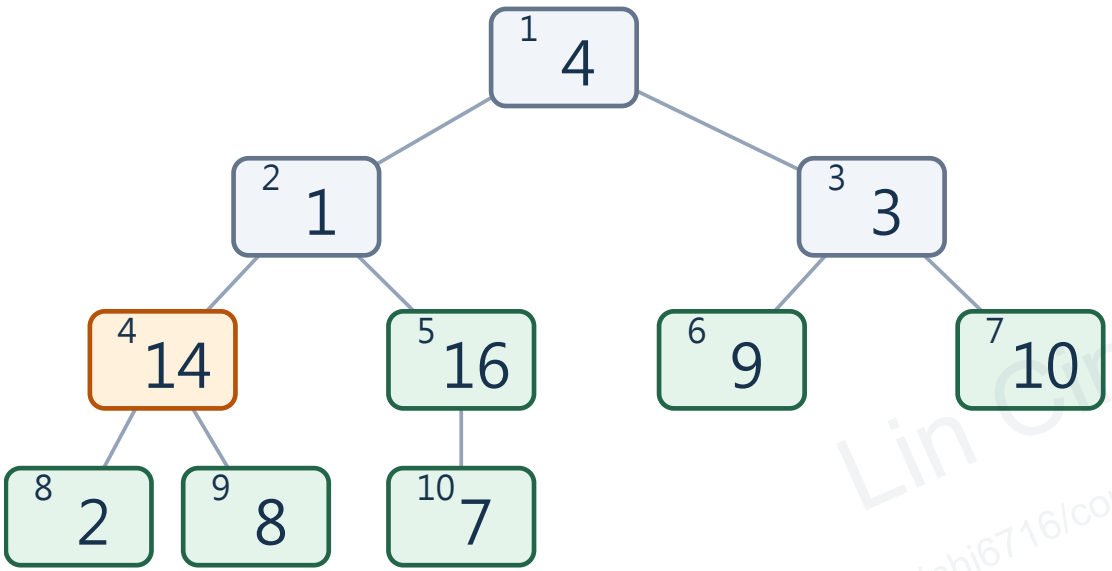
橘：本輪已完成的子樹根

綠：先前已合法；灰：待處理

位置 5 的鍵 16 與左子節點 10 的鍵 7 比較，已滿足 $16 \geq 7$，不交換。右子索引 $11 > s$，不能讀取。位置 6..10 都是葉；加上本輪完成的子樹根 5，由下往上的不變量繼續成立。

Lin Cing
http://github.com/xinhi6716/computer-science-self-study

位置四：較大的子鍵十四上移



節點左上：索引　中央：鍵值

索引固定，鍵值移動

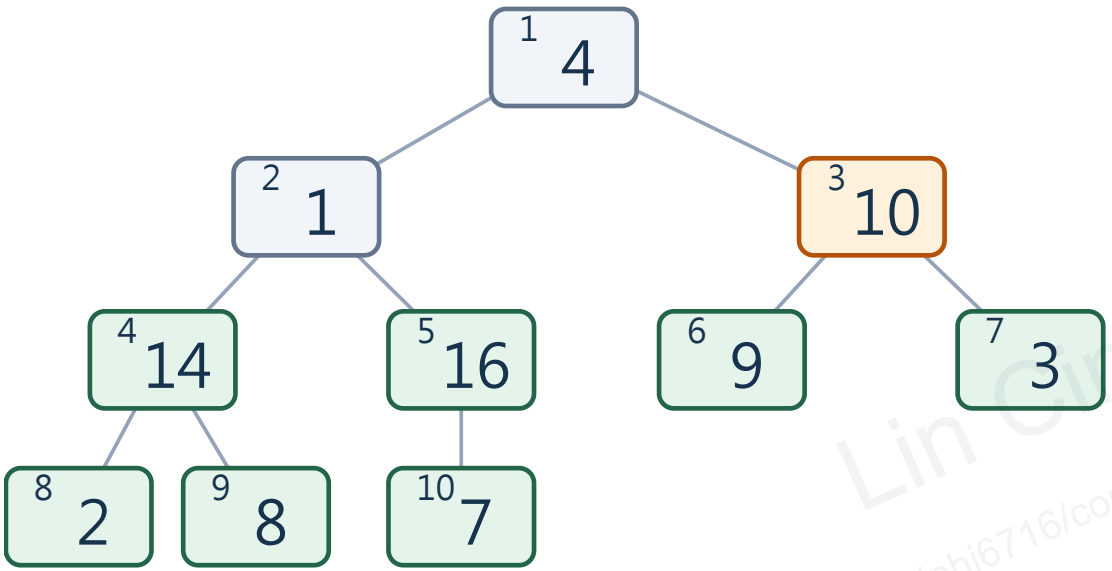
1	2	3	4	5
4	1	3	14	16
6	7	8	9	10
9	10	2	8	7

橘：本輪已完成的子樹根

綠：先前已合法；灰：待處理

位置 4 原鍵為 2，子索引 $8, 9$ 的鍵為 $14, 8$。先選較大子鍵 14，再判斷 $14 > 2$，交換位置 $4, 8$。舊鍵 2 到葉位置 8 後停止，位置 9 的鍵 8 不變。

位置三：從兩個子節點中選十



節點左上：索引　中央：鍵值

索引固定，鍵值移動

1	2	3	4	5
4	1	10	14	16
6	7	8	9	10
9	3	2	8	7

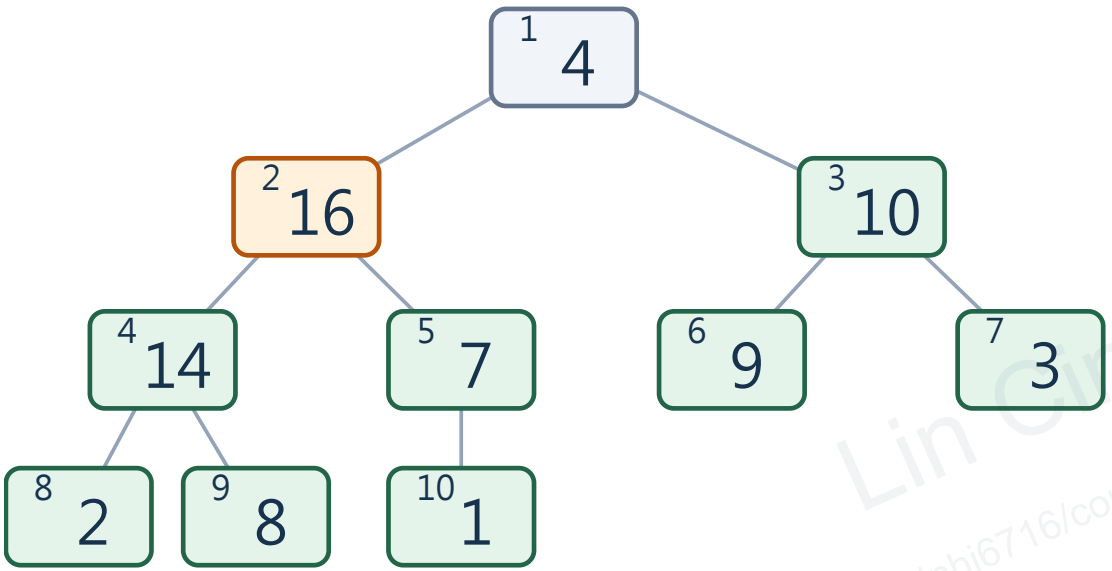
橘：本輪已完成的子樹根

綠：先前已合法；灰：待處理

位置 3 的鍵為 3，子索引 $6, 7$ 的鍵為 $9, 10$。選較大鍵 10，交換位置 $3, 7$。舊鍵 3 到葉位置 7 後停止，位置 6 的鍵 9 不變。兩個子樹原先合法，交換後以位置 3 為根的整棵子樹也合法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

位置二：舊鍵一沿五、十下沉



節點左上：索引　中央：鍵值

索引固定，鍵值移動

1	2	3	4	5
4	16	10	14	7
6	7	8	9	10
9	3	2	8	1

橘：本輪已完成的子樹根

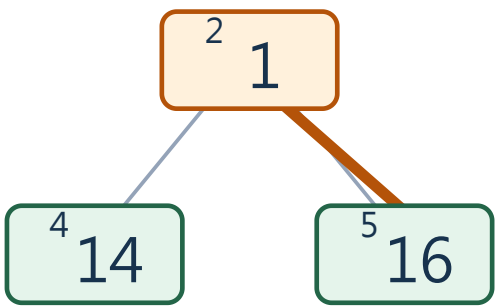
綠：先前已合法；灰：待處理

位置 2 的子鍵為 $14, 16$，先與較大的 16 交換。舊鍵 1 到位置 5 後，仍小於唯一子鍵 7，再交換位置 $5, 10$。路徑為 $2 \to 5 \to 10$；這輪不改動位置 4 的鍵 14。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

追蹤同一個舊鍵：每一步只選當前子節點

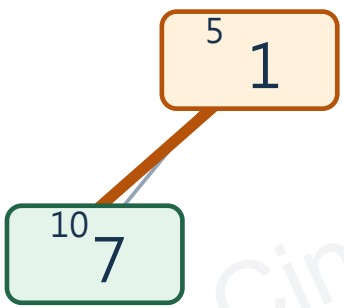
比較 1



16 > 1 → swap(2,5)

A[2]=16; A[5]=1

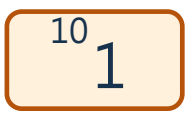
比較 2



7 > 1 → swap(5,10)

A[5]=7; A[10]=1

到葉，停止



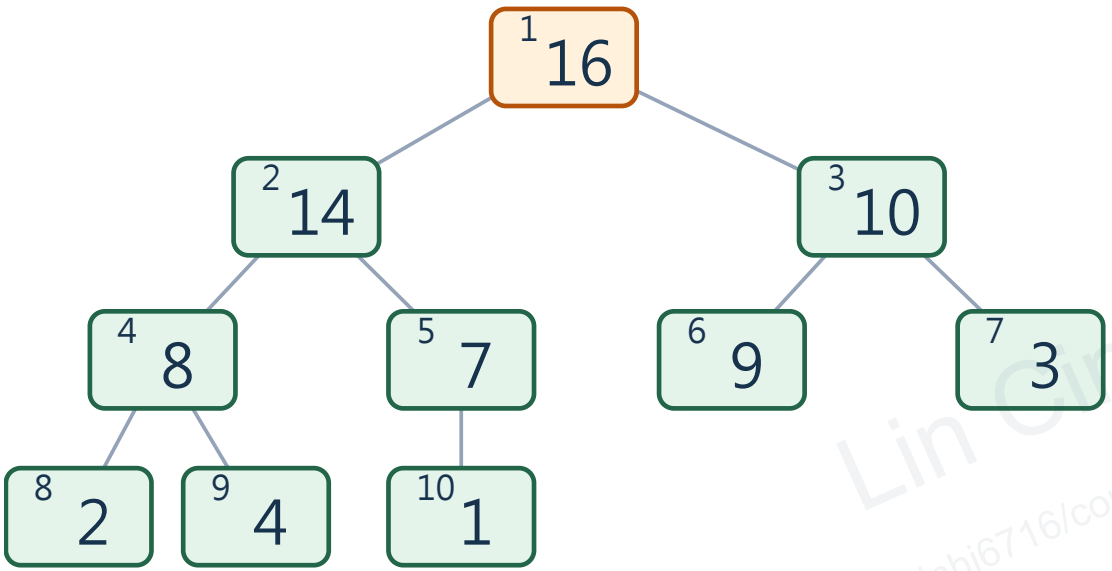
20, 21 > heap-size 10

不再讀取 child

舊鍵 1 留在索引 10

橘框始終追蹤舊鍵 1；粗線指出本次選中的較大子節點。每個節點左上角是索引、中央是鍵。第一次在子索引 4, 5 中選 5，第二次在位置 5 只有子索引 10。未畫出的子樹保持合法，不會因為未被走訪就需要重排。

位置一：根鍵四沿二、四、九下沉



節點左上：索引　中央：鍵值

索引固定，鍵值移動

1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

橘：本輪已完成的子樹根

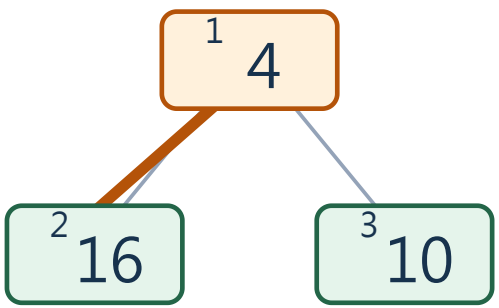
綠：先前已合法；灰：待處理

舊根鍵 4 先後與 $16, 14, 8$ 交換，停在葉位置 9。最終各非葉的父子關係為 $16 \geq 14, 10$；$14 \geq 8, 7$；$10 \geq 9, 3$；$8 \geq 2, 4$；$7 \geq 1$。這些關係涵蓋所有九條樹邊，因此整棵樹為 max-heap；十筆資料的多重集合不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

根的三次下降：索引路徑與鍵值分開看

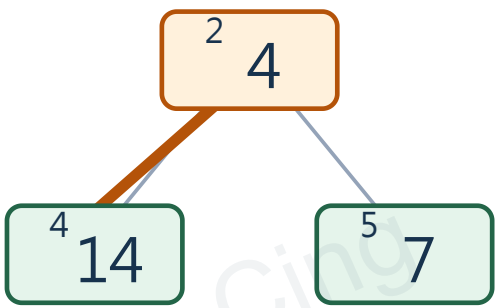
比較 1



16 > 4 → swap(1,2)

A[1]=16; A[2]=4

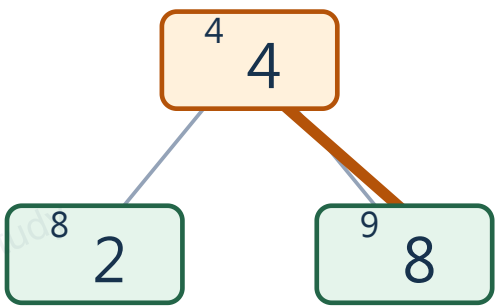
比較 2



14 > 4 → swap(2,4)

A[2]=14; A[4]=4

比較 3



8 > 4 → swap(4,9)

A[4]=8; A[9]=4

每框畫的是交換前的局部狀態，下方列出交換後的鍵值。路徑索引為 $1 \to 2 \to 4 \to 9$，同一個舊鍵始終是 4；各步選中的子鍵則為 $16, 14, 8$。到位置 9 後，子索引 $18, 19$ 都超過 $s = 10$，所以停止。

逐次交換後的完整陣列

接續位置 3 修復完成後的陣列，所有索引固定為 $1..10$：

本輪根	交換索引	交換後 $A[1..10]$
2	$2 \leftrightarrow 5$	$[4, 16, 10, 14, 1, 9, 3, 2, 8, 7]$
2	$5 \leftrightarrow 10$	$[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$
1	$1 \leftrightarrow 2$	$[16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$
1	$2 \leftrightarrow 4$	$[16, 14, 10, 4, 7, 9, 3, 2, 8, 1]$
1	$4 \leftrightarrow 9$	$[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

核對方法：每次先在目前節點的有效子節點中選最大，再判斷它是否大於父鍵。一次交換只改兩格，其餘八格與資料多重集合保持不變。要追蹤下降路徑，應保存這些中間狀態；最終陣列本身沒有記錄完整操作歷史。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性建堆：把節點高度逐層加總

先假設 $n \geq 1$。位置 i 至少高 h，當且僅當它最左的第 h 代後裔仍存在，也就是 $i2^h \leq n$。所以高度至少 h 的節點數**恰為** $\lfloor n/2^h \rfloor$。

把每個高度拆成 h 個單位，再交換有限求和次序：

$$\sum_{i=1}^{n} height(i) = \sum_{h=1}^{\lfloor \lg n \rfloor} \#\{i : height(i) \geq h\} = \sum_{h=1}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^h} \right\rfloor < n.$$

一次修復花 $O(height(i)+1)$，呼叫共 $\lfloor n/2 \rfloor$ 次，因此總成本 $O(n)$。$n \geq 2$ 時，迴圈本身至少做 $\lfloor n/2 \rfloor$ 次固定工作，給 $\Omega(n)$，合為 $\Theta(n)$。

例如 $n = 15$，高度和為 $7 + 3 + 1 = 11$；不能把每個節點都當成根的高度。空堆積的高度和為零，沒有修復呼叫，直接返回；不代入 $\lg 0$，也不聲稱 $0 < 0$。逐筆插入的新鍵可能一路上浮，其最壞總成本 $\Theta(n \lg n)$；那是另一個建堆程序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

堆積排序：有效前綴逐步縮小

```
HEAPSORT(A):
    BUILD-MAX-HEAP(A)
    for end = length(A) downto 2:
        swap(A[1],A[end])
        MAX-HEAPIFY(A,1,end-1)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

堆積排序的不變量、時間與空間

每輪開始時，$A[1..end]$ 為 max-heap；尾端 $A[end+1..n]$ 已遞增，而且尾端每個鍵都不小於堆積中的任一鍵。

交換把前綴最大鍵送到 $A[end]$，使排序尾端再長一格。有效大小改為 $end-1$，原根的兩棵子樹仍合法，所以只需修復新根。交換保留資料多重集合；終止時剩一筆前綴，整體有序。

建堆 $O(n)$，其後至多 $n-1$ 次修復各花 $O(\lg(n+1))$，故時間 $O(n\lg(n+1))$。比較排序下界給最壞 $\Theta(n\lg n)$。本節相等即停的實作，遇全相等鍵可只花 $\Theta(n)$，不能把最壞界套到每份輸入。

迭代修復使額外空間 $O(1)$；若改用遞迴修復，須另計呼叫堆疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

十筆堆積排序：九次取出最大鍵

從建堆結果 $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ 開始。每輪先交換位置 1 與舊末端 $s+1$，把最大鍵送到排序尾端，再修復新大小 s 的前綴。表中的路徑只列後續下沉，不含根與末端的交換。

新大小 s	修復路徑	修復後堆積前綴	已排序尾端
9	1→2→4→9	$[14, 8, 10, 4, 7, 9, 3, 2, 1]$	$[16]$
8	1→3→6	$[10, 8, 9, 4, 7, 1, 3, 2]$	$[14, 16]$
7	1→3→7	$[9, 8, 3, 4, 7, 1, 2]$	$[10, 14, 16]$
6	1→2→5	$[8, 7, 3, 4, 2, 1]$	$[9, 10, 14, 16]$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最後五輪：前綴縮到只剩一筆

新大小 s	修復路徑	修復後堆積前綴	已排序尾端
5	$1 \to 2 \to 4$	$[7, 4, 3, 1, 2]$	$[8, 9, 10, 14, 16]$
4	$1 \to 2$	$[4, 2, 3, 1]$	$[7, 8, 9, 10, 14, 16]$
3	$1 \to 3$	$[3, 2, 1]$	$[4, 7, 8, 9, 10, 14, 16]$
2	$1 \to 2$	$[2, 1]$	$[3, 4, 7, 8, 9, 10, 14, 16]$
1	—	$[1]$	$[2, 3, 4, 7, 8, 9, 10, 14, 16]$

例如第二輪，舊末端的鍵 1 到根後沿 $1 \to 3 \to 6$ 下沉，先與 10、再與 9 交換。每輪的尾端皆遞增，而且所有尾端鍵都不小於有效堆積的任一鍵；最後合併得到完整遞增序列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

優先權佇列：最大值、取出、升鍵與插入

$Q.A[1..Q.s]$ 是 max-heap，陣列可寫容量為 `Q.capacity`。以下固定容量程序先檢查空／滿；所有交換搬動整筆資料，鍵值只決定優先權。

```
MAXIMUM(Q):
    if Q.s == 0: return EMPTY
    return Q.A[1]

EXTRACT-MAX(Q):
    if Q.s == 0: return EMPTY
    result = Q.A[1]; Q.A[1] = Q.A[Q.s]; Q.s -= 1
    if Q.s > 0: MAX-HEAPIFY(Q.A,1,Q.s)
    return result
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

`MAXIMUM` 為 $O(1)$；`EXTRACT-MAX` 保存最大資料，以末筆補根、縮小有效大小，再向下修復，最壞 $O(\lg(s+1))$。單筆取出後大小為零，不讀新的根。

升鍵：只沿祖先方向修復

```
SIFT-UP(Q,i):
    while i > 1 and key(Q.A[floor(i/2)]) < key(Q.A[i]):
        swap(Q.A[i],Q.A[floor(i/2)])
        i = floor(i/2)

INCREASE-KEY(Q,i,k):
    require 1 <= i <= Q.s and k >= key(Q.A[i])
    set-key(Q.A[i],k)
    SIFT-UP(Q,i)
```

升鍵可能破壞的只有祖先方向：新鍵仍不小於原有子鍵。每次交換修好目前父子邊，候選鍵繼續向上；到根或父鍵不小於它時停止，最壞 $O(\lg(s+1))$。and 從左往右短路，先確認 $i > 1$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

直接插入新資料，不需要哨兵鍵

```
INSERT(Q,record):
    if Q.s == Q.capacity: return FULL
    Q.s += 1
    Q.A[Q.s] = record
    SIFT-UP(Q,Q.s)
```

直接把新資料寫在新的最後位置，再上浮，不需額外的負無窮哨兵。固定容量且未滿時，插入最壞為 $O(\lg(s+1))$；若動態陣列需要重新配置，單次搬移可能為 $\Theta(s)$，須另外分析擴容的攤銷成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從位置八開始，核對每次上浮

升鍵沿父索引 8 → 4 → 2 上浮

	1	2	3	4	5	6	7	8
原堆積	20	15	18	7	12	9	17	2
位置 8 升鍵	20	15	18	7	12	9	17	**16**
交換 8 ↔ 4	20	15	18	**16**	12	9	17	7
交換 4 ↔ 2	20	**16**	18	15	12	9	17	7

16 先後與父鍵 7、15、20 比較；最後 20 ≥ 16，停止上浮。

例中先在位置 8 寫入 16，依序比較父鍵 $7, 15, 20$；交換 $8 \leftrightarrow 4$、$4 \leftrightarrow 2$ 後停止。位置索引固定，資料身分隨交換移動。

降鍵、指定刪除與相等優先權

max-heap 的降鍵只可能破壞往子節點的邊，故向下修復。刪除位置 i 時，以末筆補洞並縮小大小；若 i 已是舊末端，直接縮小即可。

設替補鍵為 z。若 $i > 1$ 且 $z > A[parent(i)]$，則因原父鍵不小於原被刪鍵、原被刪鍵不小於其子鍵，z 也支配那些子鍵，只需上浮。否則父邊合法，只需下沉。各操作最壞 $O(\lg(s+1))$，前提是已知位置；搜尋任意鍵仍可能要 $\Theta(s)$。

若用 handle（資料身分代號）定位，交換時必須同步更新 handle 到索引的對照表，移除被刪資料的對照。陣列索引本身不會永遠指向同一筆資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

排序與佇列的同鍵次序，使用不同的序號方向

堆積排序並不穩定：$[2a, 2b, 1c]$ 可排成 $[1c, 2b, 2a]$。若要遞增排序時保留同鍵原順序，可比較 $(key, original_index)$；較晚的同鍵資料較大，會先被放到尾端。若 max-priority queue 要讓同優先權者先到先服務，則比較 $(priority, -arrival_index)$。兩者的序號方向不同，且都改變了比較與儲存規格。

同優先權資料 a, b 依序抵達，原序號分別為 $1, 2$，鍵皆為 5：

用途	a 的比較鍵	b 的比較鍵	先取出最大者	所需次序
穩定遞增排序	$(5, 1)$	$(5, 2)$	b，先放到尾端	輸出 a, b
同優先權先到先服務	$(5, -1)$	$(5, -2)$	a，立即服務	服務 a, b

遞增排序會把較大的鍵從右往左放；佇列則直接服務被取出的最大者。使用序號前，先確認「取出的先後」如何對應目標次序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雇用問題：比較成本與替換成本要分開

有 n 位能力相異、可比較排名的候選人；排名越大越好。每次面試後，只要比現任者好就立刻替換。虛擬候選人 dummy 比任何人都差。

```
HIRE(A):
    best = dummy; hires = 0
    for i = 1 to n:
        interview A[i]
        if rank(A[i]) > rank(best):
            best = A[i]
            hire best; hires += 1
    return (best, hires)
```

第 i 輪後，best 是前 i 位中能力最好的候選人：初始無人；新來者較好才替換，否則保留原最佳者。因此最後能找出全體最佳者。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一個程序，三種不同的成本問題

面試費為 c_I、每次雇用費為 c_H，雇用 X 次的總費用為 $nc_I + Xc_H$。空輸入返回 dummy、$X = 0$。

對 $n \geq 1$，排名遞增時 $X = n$，逆序時 $X = 1$。接著區分三種問法：

問法	隨機來源或量詞	要算的量
最壞情形	所有合法輸入取最大	$\max_A X(A) = n$
平均情形	明定輸入分布，例如 $n!$ 種順序等機率	$\mathbb{E}_A[X(A)]$
隨機化程序	固定任意輸入，由程序均勻洗牌	$\mathbb{E}_R[X(A, R)]$

均勻洗牌使每個固定的相異排名輸入變成均勻排列；前提是能控制處理順序。期望保證仍容許壞執行：遞增排列的機率為 $1/n!$。雇用次數少，只降低雇用費，並未減少面試次數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

指示變數：把計數拆成可以求機率的事件

先固定有限樣本空間 Ω，各結果的機率非負、總和為 1。隨機變數 X 的期望是 $\mathbb{E}[X] = \sum_{\omega \in \Omega} \Pr(\omega) X(\omega)$。

事件 E 的指示變數 I_E 在 E 成立時為 1，否則為 0，故

$$\mathbb{E}[I_E] = 1 \cdot \Pr(E) + 0 \cdot \Pr(E^c) = \Pr(E).$$

若目標計數恰為 $X = \sum_{i=1}^{m} X_i$，交換兩個有限和可得

$$\mathbb{E}[X] = \sum_{\omega} \Pr(\omega) \sum_{i} X_i(\omega) = \sum_{i} \sum_{\omega} \Pr(\omega) X_i(\omega) = \sum_{i} \mathbb{E}[X_i].$$

這是**期望線性性**，不要求 X_i 獨立。獨立性可能用在計算單項事件的機率，或將交集機率寫成乘積時。

反例：令 $X_1 = X_2$ 都是同一次公平擲幣的正面指示。仍有 $\mathbb{E}[X_1 + X_2] = 1$，但 $\Pr(X_1 = X_2 = 1) = 1/2$，不是 $1/4$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

三個機率工具：聯集界、Markov 界與尾和

先辨認正在求「事件機率」「個數的期望」或「最大值的期望」。以下三個工具都可從指示變數推出。

聯集指示滿足 $I_{\cup_i E_i} \le \sum_i I_{E_i}$，取期望即得

$$\Pr\left(\bigcup_i E_i\right) \le \sum_i \Pr(E_i).$$

若 $X \ge 0$、$t > 0$，由 $X \ge t I_{\{X \ge t\}}$ 得 Markov 界

$$\Pr(X \ge t) \le \frac{\mathbb{E}[X]}{t}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整數尾和：每個結果到達了幾個門檻

若 L 為非負整數，逐個結果皆有 $L = \sum_{j \ge 1} I_{\{L \ge j\}}$，故

$$\mathbb{E}[L] = \sum_{j \ge 1} \Pr(L \ge j).$$

有限上界時可直接交換有限和；無限尾和由非負部分和的單調極限成立，兩側亦可同為無窮大。這三個工具均不需要事件獨立。

獨立性若用在具體例子的機率乘法，仍須另外明列；不能因工具不要求獨立，就省略模型中的獨立條件。

數例：$L = 0, 1, 3$ 的機率分別為 $1/2, 1/4, 1/4$，直接算得期望 $0/2 + 1/4 + 3/4 = 1$。尾機率依序為 $\Pr(L \ge 1) = 1/2$、$\Pr(L \ge 2) = 1/4$、$\Pr(L \ge 3) = 1/4$，加總也等於 1。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

雇用次數：先算每個位置刷新紀錄的機率

假設 $n!$ 種相異排名排列等機率，令 X_i 表示第 i 位是否被雇用。固定前 i 位的候選人集合後，這些人的 $i!$ 種相對順序仍等可能，最高者落在最後位置的機率為 $1/i$。

第 i 位被雇用，恰好等價於「它是前 i 位的最高者」，所以

$$\mathbb{E}[X] = \sum_{i=1}^{n} \Pr(X_i = 1) = \sum_{i=1}^{n} \frac{1}{i} = H_n.$$

由 $1/x$ 遞減，矩形面積與積分比較給出

$$\int_{1}^{n+1} \frac{dx}{x} \le H_n \le 1 + \int_{1}^{n} \frac{dx}{x}, \qquad n \ge 1.$$

因此 $\mathbb{E}[X] = \Theta(\ln n)$，而總期望費用仍為 $nc_I + c_H H_n$；面試費這一項並未變成對數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

六人逐輪雇用，與同規模期望值對照

位置 i	排名	面試後最佳排名	本次雇用？	累積次數
1	2	2	是	1
2	5	5	是	2
3	1	5	否	2
4	4	5	否	2
5	6	6	是	3
6	3	6	否	3

數例 $[2, 5, 1, 4, 6, 3]$ 在位置 $1, 2, 5$ 刷新紀錄，共雇用三次；同規模平均為 $H_6 = 49/20 = 2.45$。只有一次雇用的機率為 $1/n$；每位都雇用只有一種遞增排列，機率為 $1/n!$。空輸入另取 $H_0 = 0$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一個方法，也能算逆序對與拿回原帽的人數

均勻排列 n 個相異鍵。對位置 $i < j$，令 I_{ij} 表示 $A[i] > A[j]$。交換兩位置的值，將逆序與非逆序結果一對一配對，因此 $\Pr(I_{ij} = 1) = 1/2$。

$$V = \sum_{i<j} I_{ij}, \qquad \mathbb{E}[V] = \binom{n}{2}\frac{1}{2} = \frac{n(n-1)}{4}.$$

共享位置的逆序事件可能相依；這不影響期望線性性。當 $n = 2$，逆序數只能是 0 或 1，但期望為 $1/2$。

另一例：將 $n \geq 1$ 頂帽子均勻地一對一還給原主人們。第 i 人拿回自己的帽子的機率為 $1/n$，固定點數 F 滿足

$$F = \sum_{i=1}^{n} I_i, \qquad \mathbb{E}[F] = n\frac{1}{n} = 1.$$

$n = 2$ 時，可能兩人都拿對或都拿錯，各有 $1/2$ 機率。期望為一人，不表示一定恰有一人拿對。$n = 0$ 時直接有 $F = 0$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

均勻整數抽樣：拒絕超出範圍的位元結果

RANDOM(a,b) 的契約是回傳包含兩端的整數，各機率 $1/(b-a+1)$，且新呼叫條件於歷史仍均勻。以下以公平、獨立的位元實作；必須有整數 $a \le b$。

令 $m = b - a + 1$。$m = 1$ 直接回傳 a；否則令 $k = \lceil \lg m \rceil$，反覆讀取 k 個位元作為 $z \in \{0, \dots, 2^k - 1\}$。若 $z < m$，回傳 $a + z$；否則重抽整組。

每輪接受率 $p = m/2^k > 1/2$。對任一合法值 v，回傳它的機率為

$$\sum_{t=0}^{\infty} (1-p)^t \frac{1}{2^k} = \frac{1}{p 2^k} = \frac{1}{m}.$$

重試輪數期望為 $1/p < 2$，因此期望位元成本為 $O(\lg m)$。拒絕抽樣幾乎必然終止，沒有預先固定的重試次數上限。

直接取模會產生偏差：八個三位元結果對 3 取模，餘數 $0, 1, 2$ 的前像數為 $3, 3, 2$，不是等機率。偽亂數產生器與整數取樣方式，也須分別滿足所採模型。

原地洗牌：每輪多固定一個位置

每輪從尚未固定的後綴（suffix）中均勻選一筆，與位置 i 交換。即使鍵相同，也把每筆資料視為不同身分。

```
SHUFFLE(A):
    for i = 1 to n:
        j = RANDOM(i,n)
        swap(A[i],A[j])
```

第 i 輪後，後續交換不再碰 $A[1..i]$。空輸入只有一種空排列；單筆輸入只與自己交換。單位成本整數抽樣模型下，時間 $\Theta(n)$、額外空間 $O(1)$；位元抽樣成本另計。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

均勻洗牌：逐輪延伸指定的前綴

主張：第 i 輪前，任一指定的 $i-1$ 個不同身分序列占據 $A[1..i-1]$ 的機率為 $(n-i+1)!/n!$。初始 $i=1$，空前綴的機率為一。

固定此前綴後，下一個指定身分恰占剩餘 $n-i+1$ 格中的一格。條件均勻抽樣以 $1/(n-i+1)$ 的機率選中它，因此指定長 i 前綴的機率為

$$\frac{(n-i+1)!}{n!}\,\frac{1}{n-i+1}=\frac{(n-i)!}{n!}.$$

歸納至 $i=n$，每個完整排列的機率皆為 $0!/n!=1/n!$。取 $n=4$、指定輸出 $[C,D,A,B]$：

固定長度	0	1	2	3	4
指定前綴	空	C	C,D	C,D,A	C,D,A,B
機率	1	$1/4$	$1/12$	$1/24$	$1/24$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

四筆洗牌：逐輪檢查固定前綴

從 $[A, B, C, D]$ 開始，依序抽到 $j = 3, 4, 3, 4$。字母是身分，不是比較用的鍵。

輪次 i	合法索引	抽到 j	交換後完整陣列	已固定前綴
1	$1..4$	3	$[C, B, A, D]$	C
2	$2..4$	4	$[C, D, A, B]$	C, D
3	$3..4$	3	$[C, D, A, B]$	C, D, A
4	4	4	$[C, D, A, B]$	C, D, A, B

此路徑機率為 $(1/4)(1/3)(1/2)(1) = 1/24$。任意指定輸出也唯一決定一條合法路徑：依次在剩餘後綴找到指定的下一個身分，再將它換到位置 i。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

已固定的前綴，後面不再移動

從 [A,B,C,D] 開始：每輪只從尚未固定的 suffix 選一個

	位置1	位置2	位置3	位置4	本輪機率
i=1, j=3	C	B	A	D	1/4
i=2, j=4	C	D	A	B	1/3
i=3, j=3	C	D	A	B	1/2
i=4, j=4	C	D	A	B	1

綠格已固定、不再改動；這條路徑機率 $1/4 \cdot 1/3 \cdot 1/2 \cdot 1 = 1/24$。

後面的輪次只交換尚未固定的位置；這是前綴能永久固定的原因。單次追蹤用來核對程序，一般均勻性仍由前面的聯合不變量證明。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

每個位置看似均勻，整個排列仍可能有偏

隨機選一個循環平移量，確實能讓每筆以 $1/n$ 機率落在每個位置；但 $n \geq 3$ 時只有 n 種可能輸出，而非 $n!$ 種。均勻邊際分布不等於聯合均勻。

另一個錯誤程序讓 $i = 1..n$，每輪都從整個 $1..n$ 抽 j 再交換。$n = 3$ 時有 $3^3 = 27$ 條等可能亂數路徑；若六種排列均勻，每種須對應 $27/6$ 條路徑，但路徑數必為整數，故不可能。

錯誤程序會重新移動先前的位置，因此不能沿用「已固定前綴」的證明。證明某個程序不均勻，與只說「看起來不夠隨機」不同。

正確洗牌若只做前 m 輪，$0 \leq m \leq n$，每個有序樣本的機率為 $(n-m)!/n!$；忽略順序後，每個 m 元子集合有 $m!$ 種排列，機率為 $1/\binom{n}{m}$。這省下後續交換；若輸入必須先建成可索引陣列，建表成本仍要計入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機優先值：沒有碰撞時才直接得到均勻排列

給 $n \geq 1$ 筆資料獨立均勻的優先值 $P_i \in \{1, \ldots, M\}$，其中整數 $M \geq n$，再按優先值排序。條件於所有優先值不同，交換資料標籤會一對一對應不同次序，且保留機率，因此 $n!$ 種相對順序等可能。

每對碰撞機率為 $1/M$。聯集界（union bound）給

$$\Pr(\exists i < j : P_i = P_j) \leq \frac{\binom{n}{2}}{M}.$$

取 $M = n^3$、$n \geq 2$，上界為 $(n-1)/(2n^2) < 1/(2n)$。碰撞很少，仍不等於不會碰撞。

若平手時保留原順序，無條件輸出有偏：$n = 2$ 時，第一筆先出的機率為 $\Pr(P_1 \leq P_2) = (M+1)/(2M) > 1/2$。

可在任何碰撞時整組重抽，再接受無碰撞結果；上述 M 下接受率有正常數下界，期望輪數為常數。也可在每個平手群中獨立均勻洗牌。用比較排序實作優先值排序需 $\Theta(n \lg n)$ 時間與 $O(n)$ 優先值空間；重抽另須偵測碰撞，不能當成免費。

隨機化之後，仍要說清楚正確性與可控制的部分

RANDOMIZED-HIRE 先均勻洗牌，再執行 HIRE。對每個固定的相異排名輸入，洗牌後皆為均勻排列，故期望雇用數為 H_n，期望費用為 $nc_I + c_H H_n$。另有 $\Theta(n)$ 洗牌操作，不能把操作時間與雇用費用混成同一單位。

若候選人只能依外部順序到達、不能重新安排，就沒有這個洗牌控制權。此時 H_n 的分析仍須對外部順序假設均勻，不能用一個無法執行的步驟取消輸入假設。

本章的洗牌與隨機 pivot 排序，隨機的是成本；每次返回都符合規格。這類稱 Las Vegas 演算法。Monte Carlo 演算法允許小機率錯答，必須另列錯誤機率與其適用輸入；只給期望時間不能推得正確率。

後面的線上遴選又是不同目標：策略可能沒有選到全域最佳，但仍按規定選出一人。它的成功機率要依遴選模型另證，不能與排序的正確性保證混用。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

快速排序：分割後，兩側仍要各自排序

本節使用 1 起算閉區間 $A[p..r]$。選定 pivot（分割基準值）後，分割只建立它與兩側的大小關係，還沒有把兩側內部排好。

```
QUICKSORT(A,p,r):
    if p >= r: return
    q = PARTITION-LAST(A,p,r)
    QUICKSORT(A,p,q-1)
    QUICKSORT(A,q+1,r)
```

空區間 $p > r$、單筆區間 $p = r$ 都直接返回。分割回傳 pivot 的位置 q；兩個遞迴區間都**排除 q**，所以長度嚴格縮小。這個回傳契約是排序程序正確與終止的前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

尾端分割：保存鍵值，交換整筆資料

尾端版本的分割程序如下。鍵值 x 先保存；交換搬動整筆資料，其他欄位隨鍵一起移動。

```
PARTITION-LAST(A,p,r):
    x = key(A[r]); i = p-1
    for j = p to r-1:
        if key(A[j]) <= x:
            i += 1
            swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

迴圈邊界：i 是小鍵區尾端，j 是下一筆待分類

每輪開始時，pivot 固定在 A[r]。本章採 **1 起算索引、閉區間 A[p..r]**；i 可以等於 p−1，表示 ≤ 區為空。

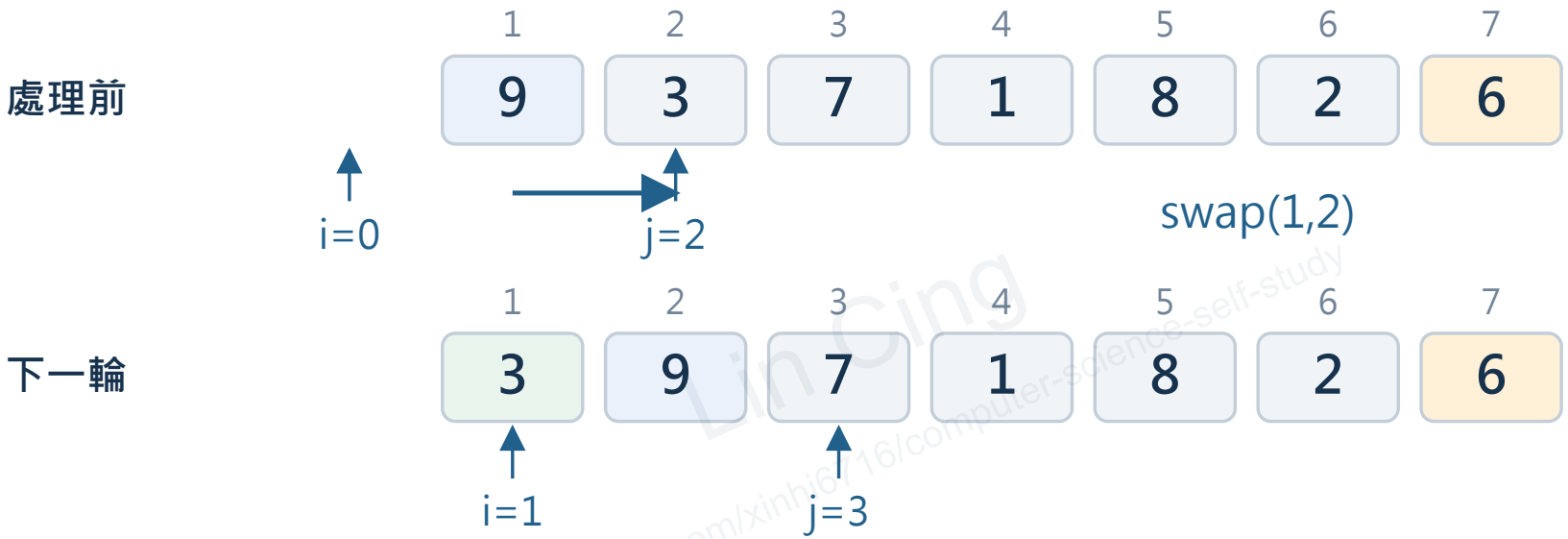
區域	已知關係	初始狀態 i=p−1、j=p
A[p..i]	≤ pivot	空
A[i+1..j−1]	> pivot	空
A[j..r−1]	尚未分類	除 pivot 外全部
A[r]	pivot	全程固定到收尾交換

讀圖方式： 綠色是 ≤ 區，藍色是 > 區，灰色是未知區，橘色是 pivot。以下兩頁都沿用 [9,3,7,1,8,2,6]、pivot=6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

小鍵到來：交換後，兩個邊界都前進

3 ≤ 6：先讓 i 前進，再交換 A[1] 與 A[2]

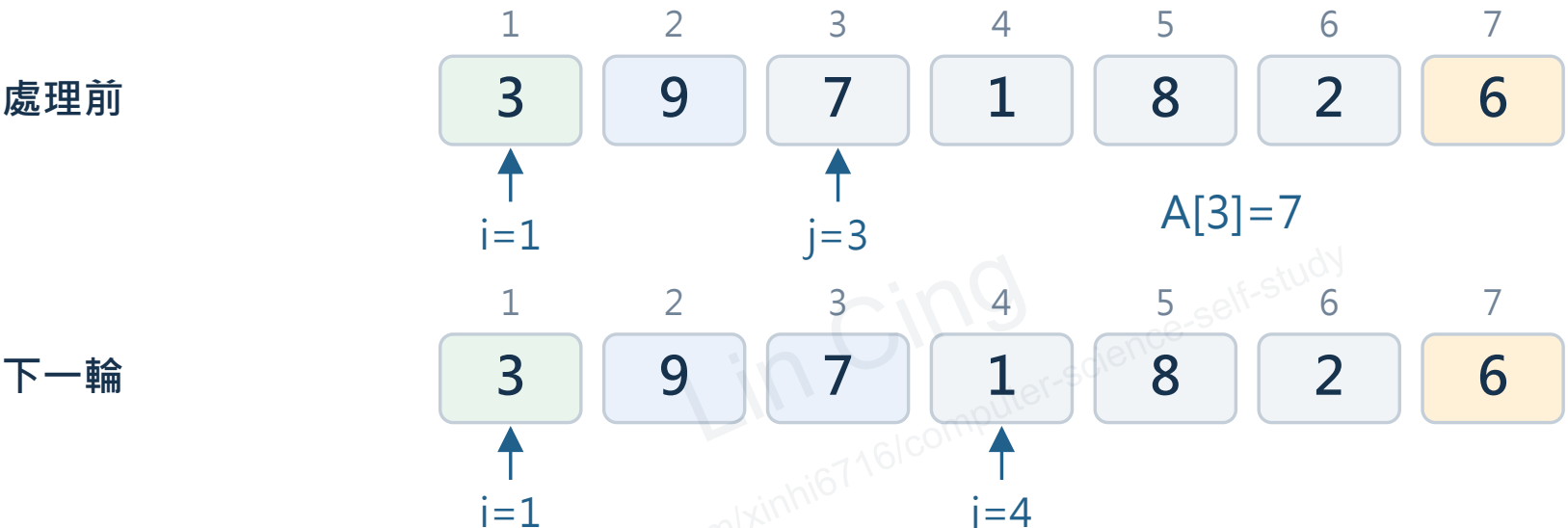


3 加入 ≤ 區；9 回到 > 區。pivot 6 仍在索引 7。

處理 A[2]=3 時，先令 i=1，再交換 A[1]、A[2]。3 加入 ≤ 區，原先的 9 留在 > 區；最後 j 前進。一般情況也可能 i=j，此時是自己與自己交換。

大鍵到來：不搬資料，只擴大已分類區

7 > 6：不交換、不改 i，只讓 j 前進

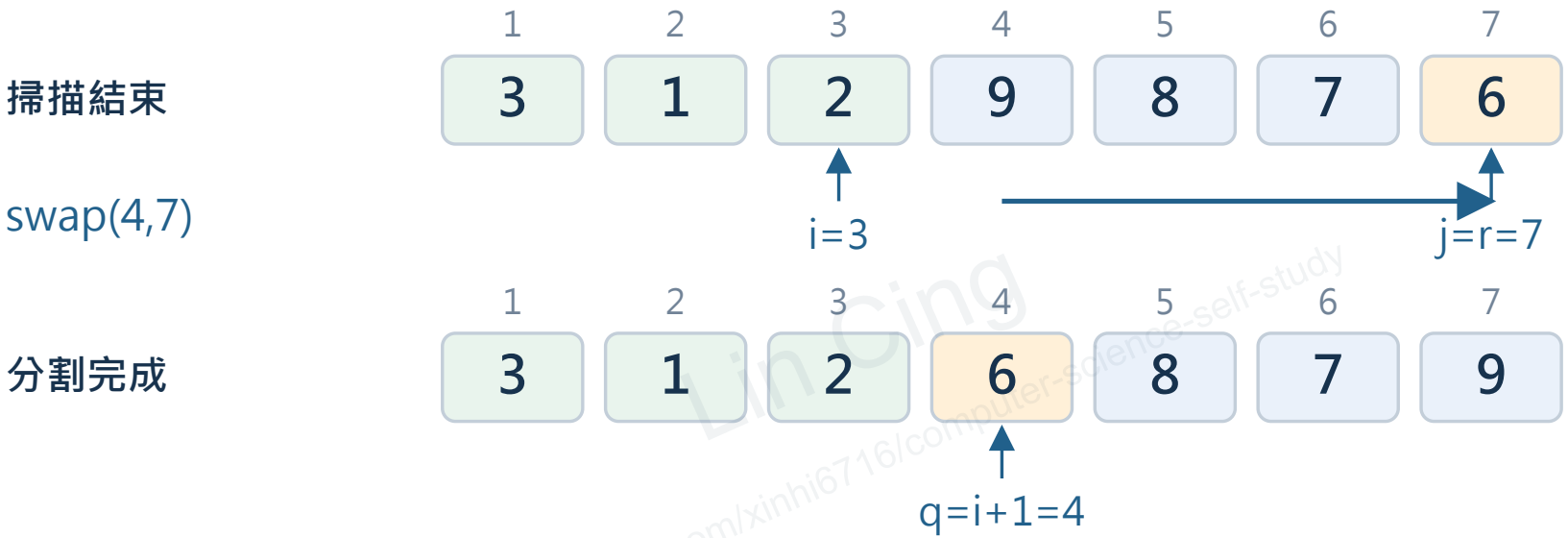


7 從未知區加入 > 區；陣列值不變，但分類邊界改變。

A[3]=7>6，因此 i 不變、陣列不變，只把 j 往右移一格。**不變的是值，改變的是已知資訊：** 7 現在屬於 > 區，不再是未知值。

收尾交換：q 才是 pivot 的最終位置

掃描結束後，pivot 才移到兩區之間



≤6：[3,1,2] | pivot 6 | >6：[8,7,9]；兩側仍未排序。

每輪未知區長 r−j 減一，所以掃描必終止。結束時 j=r；交換 A[i+1] 與 A[r]，得到 q=i+1。左區 ≤ pivot、右區 > pivot，兩側內部仍未排序。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

從分割不變量到整體排序正確性

1. **初始化：** i=p−1、j=p，前兩區都是空區間，不變量成立。
2. **保持：** 小鍵交換後擴大 ≤ 區；大鍵只擴大 > 區。兩種情況都使未知區少一筆，pivot 仍固定。
3. **分割終止：** 未知區消失，收尾交換把 pivot 放到 q。交換只重排元素，不會遺失或新增資料。
4. **遞迴歸納：** 空區間、單元素已排序。兩個子區間排除 q，長度都變小；歸納假設使它們各自有序，再接上 pivot，整個區間便有序。

只檢查「左小右大」還不夠：完整正確性同時需要**元素多重集合不變、遞迴嚴格縮小**，以及兩側遞迴回來後確實有序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整分割追蹤：每次比較與交換都列出

輸入 $A = [9, 3, 7, 1, 8, 2, 6]$、$p = 1, r = 7$。尾端鍵 $x = 6$，初始 $i = 0$；表中 i 是本輪判斷後的值。

掃描索引 j	判斷	i	操作後的完整陣列
1	$9 > 6$，不交換	0	$[9, 3, 7, 1, 8, 2, 6]$
2	$3 \le 6$，交換 $1, 2$	1	$[3, 9, 7, 1, 8, 2, 6]$
3	$7 > 6$，不交換	1	$[3, 9, 7, 1, 8, 2, 6]$
4	$1 \le 6$，交換 $2, 4$	2	$[3, 1, 7, 9, 8, 2, 6]$
5	$8 > 6$，不交換	2	$[3, 1, 7, 9, 8, 2, 6]$
6	$2 \le 6$，交換 $3, 6$	3	$[3, 1, 2, 9, 8, 7, 6]$
掃描結束	交換 $4, 7$	3	$[3, 1, 2, 6, 8, 7, 9]$

回傳 $q = i + 1 = 4$。左側 $[3, 1, 2] \le 6$、右側 $[8, 7, 9] > 6$，兩側都還沒排好。掃描恰做 $r - p = 6$ 次鍵比較；連同收尾，時間 $\Theta(r - p + 1)$、額外空間 $O(1)$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

最壞時間：一整族輸入達到平方成本

已排序的 n 個相異鍵，每次取最後位置作 pivot。pivot 永遠最大，子問題長度是 n−1 與 0。

本輪長度	本輪鍵比較	下一輪非空長度
n	n−1	n−1
n−1	n−2	n−2
…	…	…
2	1	1

比較總數為 $n(n-1)/2$，給出最壞時間的 Ω(n²) 下界。這個結論適用整個輸入族，並非由單次實測推論。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

平方上界必須涵蓋每一種分割

令 C(n) 是 n 個鍵的最壞比較數，子問題大小 a+b=n−1。以較小規模的歸納假設可得

$$C(a)+C(b)+(n-1) \le \frac{a(a-1)}{2}+\frac{b(b-1)}{2}+n-1 = \frac{n(n-1)}{2}-ab \le \frac{n(n-1)}{2}.$$

C(0)=C(1)=0，故上界涵蓋所有分割；a=0 或 b=0 時恰達界。每個被選作 pivot 的元素不再進入遞迴，所以呼叫總數 O(n)。總時間 O(n+C)，配上下界得到**最壞 $\Theta(n^2)$**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

均分時，先按層數清楚計算比較工作

取 n=15，每次選真正的中位數。此表只計 Lomuto 的鍵比較，單元素不呼叫 partition。

深度 d	子問題個數	每個長度	本層鍵比較
0	1	15	14
1	2	7	2×6=12
2	4	3	4×2=8
3	8	1	0

共 34 次；同樣 15 個鍵，端點分割則做 105 次。差別來自每層保留下來的子問題大小，與 partition 本身仍是線性掃描並不矛盾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不必完全平分，固定比例也能保證對數深度

若每次非空子問題的較大側至多為原長度的 ρ 倍，其中 ρ<1 是固定常數，最長分支深度為 O(log n)。同一層的子區間互不重疊，本層總掃描量至多 n，因此總時間 O(n log n)；固定的 1:9 分割也符合。

完全均分且 $n = 2^k - 1$ 時，深度 d 有 2^d 個長度 $2^{k-d} - 1$ 的子問題。本層比較數為

$$2^d(2^{k-d} - 2) = n + 1 - 2^{d+1}, \qquad 0 \le d \le k - 2.$$

前約 k/2 層，每層仍做 Θ(n) 次比較，故這種執行本身就需 Ω(n log n)，合為 Θ(n log n)。這裡的下界來自實際工作量；比較排序的最壞輸入下界並不保證每個輸入都一樣慢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

重複鍵：隨機 pivot 不能修好全相等輸入

對 $[5,5,5,5,5]$，每個 `<= x` 都成立，故每輪 $q = r$。子問題長度依序為 $5,4,3,2$，比較次數 $4+3+2+1=10$；一般 n 筆為 $n(n-1)/2$。

隨機換來的 pivot 仍是 5，不能改變這條執行路徑。因此本節稍後對二路版本推導的期望 $\Theta(n \lg n)$，先限定**鍵互異**。本章次序統計量節會完整給出三路分割：小於、等於、大於 pivot；只有兩個嚴格區繼續遞迴。全相等時只分割一次，總時間 $\Theta(n)$。

穩定性是另一項保證。以身分區別輸入 $[2_a, 2_b, 1_c]$，尾端 pivot 為 1，收尾交換得到 $[1_c, 2_b, 2_a]$；兩筆鍵 2 的順序已反轉。隨機挑 pivot 並不會使這個交換式實作自動變成穩定排序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

只遞迴較小側：控制堆疊，沒有省掉分割

原本兩側都遞迴的程序，最壞有 $\Theta(n)$ 層堆疊；分割原地進行，不代表整個快速排序只用常數空間。可改成只遞迴較小側，較大側留在本層迴圈處理。

```
QUICKSORT-SMALL-STACK(A,p,r):
    while p < r:
        q = PARTITION-LAST(A,p,r)
        if q-p < r-q:
            QUICKSORT-SMALL-STACK(A,p,q-1)
            p = q+1
        else:
            QUICKSORT-SMALL-STACK(A,q+1,r)
            r = q-1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

較小側遞迴，限制尚未返回的呼叫鏈

兩側長度 $a + b = n - 1$，真正遞迴的一側至多 $\lfloor (n-1)/2 \rfloor$。沿尚未返回的呼叫鏈，規模至少減半，故堆疊為最壞 $O(\lg(n+1))$。

若 pivot 每次都是最大值，遞迴側為空，迴圈仍依次分割長度 $n, n-1, \ldots, 2$。此時堆疊甚至為常數，時間卻仍是 $\Theta(n^2)$；空間與時間要分別計算。

每次選中位數：一條尚未返回的呼叫鏈

巢狀框架



15 → 7 → 3 → 1：同時存活四個框架

每次選最大值：同一框架的連續迴圈

同一工作框架



每輪的空遞迴呼叫立即返回；較大側留在同一框架中，仍須執行下一次分割。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機化保證的是每個固定輸入的期望成本

每輪在目前閉區間 $p..r$ 均勻抽一個整數 k，交換 $A[k]$ 與 $A[r]$，再用同一個尾端分割。未來選擇條件於當時狀態仍須均勻。

鍵互異時，不論目前排列如何，每個局部秩成為 pivot 的機率都是 $1/n$。接下來取期望的隨機來源是**演算法內部選擇**，不要求使用者提供均勻排列。

任何固定輸入仍可能遇到每次都選端點的亂數路徑，花費 $\Theta(n^2)$。期望界既不等於每次執行的上界，也不能直接當成尚未證明的尾機率界。

數學模型的均勻抽樣須與實作分開。固定 seed 可重現某條偽亂數序列，但「程式呼叫了 random」本身不足以證明抽樣均勻；有界整數抽樣可使用本章前面的拒絕抽樣程序。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

比較一對鍵，關鍵在秩區間內的第一個 pivot

將相異鍵依大小命名為 $z_1 < \cdots < z_n$。令 X_{ij} 表示 z_i, z_j 是否直接做過一次鍵比較。它們至多比較一次：一次比較必有一方為 pivot，而 pivot 隨即退出遞迴。

考慮 $Z_{ij} = \{z_i, \ldots, z_j\}$。區間外的 pivot 把這整段留在同一側；區間內的 pivot 若是端點 z_i 或 z_j，本次便比較兩者；若是內部鍵，兩者分居不同遞迴區，往後不再相遇。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

區間外的 pivot，不會分開這一對鍵

分析排序後的秩：$Z_{2,5} = \{2,3,4,5\}$；不是陣列快照



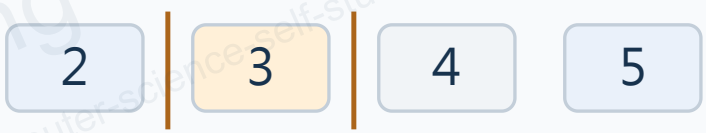
先選 7：2..5 一起留在左子問題，$X_{2,5}$ 尚未決定。

區間內先選端點 2（或 5）



該次比較 2 與 5 → $X_{2,5}=1$

區間內先選內部 3（或 4）



2 在左、5 在右 → 永不比較，$X_{2,5}=0$

四個區間鍵等可能先選；其中兩個是端點。$\Pr(X_{2,5}=1)=2/4=1/2$。

因此 $X_{ij} = 1$，恰好等價於「Z_{ij} 內第一個被選為 pivot 的鍵是兩個端點之一」。它不必是全程第一個 pivot。

從四種首選情況，推到一般比較機率

追蹤 z_2, z_5，只看 $Z_{2,5} = \{z_2, z_3, z_4, z_5\}$ 內誰先成為 pivot。

區間內第一個 pivot	z_2, z_5 的去向	$X_{2,5}$
z_2	本次直接比較	1
z_3	分居兩側，不再相遇	0
z_4	分居兩側，不再相遇	0
z_5	本次直接比較	1

四個鍵各有 $1/4$ 機率成為區間內首選，因此比較機率為 $1/2$。一般區間長 $m = j - i + 1$，故

$$\Pr(X_{ij} = 1) = \frac{2}{m} = \frac{2}{j - i + 1}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

證明區間內第一個 pivot 確實均勻

等可能性需要證明。固定區間 Z 的 m 個鍵及其中一個指定鍵 z，對目前包含 Z 的子問題大小 N 歸納。$N = m$ 時，首個 pivot 均勻，指定鍵機率 $1/m$。$N > m$ 時，本次直接選到 z 的機率 $1/N$；選到其餘 Z 內鍵則失敗。

若先選到 Z 外鍵，含 Z 的後續子問題嚴格縮小，依歸納指定鍵首選機率仍是 $1/m$。對 $N - m$ 個外部鍵求和得

$$\frac{1}{N} + \frac{N - m}{N}\frac{1}{m} = \frac{1}{m}.$$

因此區間內首選確實均勻；分母是 m，不是原輸入長度 n。其他遞迴分支可穿插執行，但不會移動或比較此子區間內的鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-s

期望比較數：把秩區間長度逐項加總

總鍵比較數 $C = \sum_{1 \le i < j \le n} X_{ij}$。期望線性性不需要這些事件獨立。長度 $k = j - i + 1$ 的秩區間共有 $n + 1 - k$ 個，因此

$$\begin{aligned} \mathbb{E}[C] &= \sum_{i<j} \frac{2}{j - i + 1} = 2 \sum_{k=2}^{n} \frac{n + 1 - k}{k} \\ &= 2(n+1) \sum_{k=2}^{n} \frac{1}{k} - 2(n-1) \\ &= 2(n+1) H_n - 4n, \qquad H_n = \sum_{k=1}^{n} \frac{1}{k}. \end{aligned}$$

由 $\ln(n+1) \le H_n \le 1 + \ln n$，得到期望比較數 $\Theta(n \lg n)$。每輪掃描工作與本輪鍵比較次數同階，所有呼叫另有 $O(n)$ 開銷，所以期望時間也是 $\Theta(n \lg n)$。

此精確式適用鍵互異、各輪條件均勻、每個非 pivot 鍵只做一次 `<=` 的版本。它不計索引測試、交換或產生亂數的成本。比較事件彼此相依，不能據此直接套用獨立變數的變異數或尾界公式。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

首端 pivot 的分割：先看它自己的不變量

以下是首端 pivot 的單向掃描版本；閉區間仍是 $A[p..r]$，但 $x = A[p]$ 固定到收尾。

```
PARTITION-FIRST(A,p,r):
    x = A[p]; i = p
    for j = p+1 to r:
        if key(A[j]) <= key(x):
            i = i+1
            swap(A[i],A[j])
    swap(A[p],A[i])
    return i
```

每輪開始時 $A[p] = x$、$A[p+1..i] \le x$、$A[i+1..j-1] > x$，$A[j..r]$ 尚未分類。小鍵先擴大左區再交換，大鍵只移動掃描邊界；未知區每輪少一筆。

終止時交換 $A[p]$ 與 $A[i]$，回傳位置 i。排序遞迴仍須排除該位置。不能把首端版本的初始 $i = p$，直接抄進尾端版本的 $i = p - 1$ 規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一個 pivot，兩種分割都能核對

首端版輸入 $[6, 10, 13, 5, 8, 3, 2, 11]$，取 $x = 6$、初始 $i = 1$。

j	判斷與操作	操作後陣列
2	$10 > 6$，$i = 1$	$[6, 10, 13, 5, 8, 3, 2, 11]$
3	$13 > 6$，$i = 1$	同前
4	$5 \le 6$，$i = 2$，交換 $2, 4$	$[6, 5, 13, 10, 8, 3, 2, 11]$
5	$8 > 6$，$i = 2$	同前
6	$3 \le 6$，$i = 3$，交換 $3, 6$	$[6, 5, 3, 10, 8, 13, 2, 11]$
7	$2 \le 6$，$i = 4$，交換 $4, 7$	$[6, 5, 3, 2, 8, 13, 10, 11]$
8	$11 > 6$，$i = 4$	同前
結束	交換 $1, 4$，回傳 4	$[2, 5, 3, 6, 8, 13, 10, 11]$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

對照兩版的相同 pivot

同一個 pivot 鍵，區內排列可以不同

	1	2	3	4	5	6	7	8
首端版	2	5	3	6	8	13	10	11

	1	2	3	4	5	6	7	8
尾端版	5	3	2	6	8	10	13	11

兩版皆 q=4、pivot=6；左側 ≤6、右側 >6，區內不必排序。

尾端版若也要選同一個 6，須先交換首尾，再分割 $[11, 10, 13, 5, 8, 3, 2, 6]$，可得 $[5, 3, 2, 6, 8, 10, 13, 11]$。兩版都把 6 放在位置 4，各做 7 次掃描鍵比較；兩側的內部排列可以不同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

名稱相近時，對照前提、邊界與回傳值

版本	pivot 與初始邊界	回傳與遞迴
CLRS 尾端單向掃描	$x = A[r]$，$i = p - 1$，$j = p$	回傳單點 q；遞迴 $[p, q-1]$、$[q+1, r]$
MIT 2005 首端單向掃描	$x = A[p]$，$i = p$，$j = p + 1$	回傳單點 q；仍排除 q
首端雙向掃描、最後定位	$x = A[p]$，左右掃描遇等值即停	指標交錯後交換 pivot，回傳單點 q
本章三路分割	先保存 pivot 鍵 x，$lt = i = p$，$gt = r$	回傳等值閉區間 $[lt, gt]$；整段排除

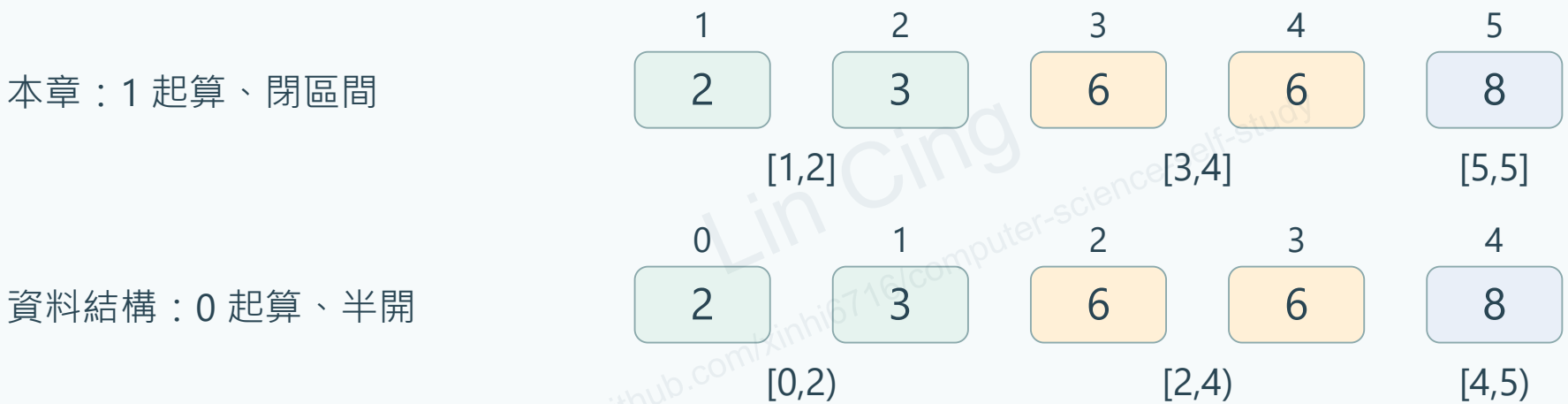
雙向掃描仍須再看回傳契約：經典 Hoare partition 回傳分界線，不保證 pivot 已在最終位置；另一種雙向版本會在收尾交換 pivot，回傳定位點。兩者的遞迴範圍不同，不能只憑「Hoare」名稱照搬。後面的雙向定位程序會明定後者的規格。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一批資料，兩套區間表示

先轉換索引與區間規格，再比較程式。對同一批資料 $[2, 3, 6, 6, 8]$，等值區在 1 起算寫成閉區間 $[3, 4]$，在 0 起算寫成半開區間 $[2, 4)$；兩者都包含兩筆資料。

同樣五筆資料，換一套索引與區間規格



只換索引表示，不移動資料；中間兩筆等值資料都排除於下一輪遞迴。

非空閉區間 $[p, r]$ 對應半開區間 $[p-1, r)$，後者不包含右端點。因此閉區間 $[p, q-1]$ 對應半開區間 $[p-1, q-1)$；兩套規格中的空區間長度都為零。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

雙向掃描後定位：遇到等值就停下來

以下使用閉區間 $p \le r$，首端 pivot。and 必須由左往右短路求值，先檢查邊界才讀陣列。

```
PARTITION-INWARD-PLACE(A,p,r):
    x = key(A[p]); i = p+1; j = r
    while i <= j:
        while i <= j and key(A[i]) < x: i += 1
        while i <= j and key(A[j]) > x: j -= 1
        if i >= j: break
        swap(A[i],A[j])
        i += 1; j -= 1
    swap(A[p],A[j])
    return j
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向掃描：不變量與完整交換追蹤

已分類左段 $A[p+1..i-1] \le x$，右段 $A[j+1..r] \ge x$。兩側掃描分別略過嚴格小、嚴格大的鍵；未交錯時交換停住的兩筆，再同時推進。等值鍵不能讓指標永久停住。未知區每輪縮小，最後 pivot 交換到 j；回傳兩側為 $[p, j-1]$、$[j+1, r]$。

輸入 $[6, 10, 13, 5, 8, 3, 2, 11]$，$x = 6$。表中索引是兩側掃描停住、尚未交換的狀態。

停住的 (i, j)	交換位置	操作後的完整陣列
$(2, 7)$	$2, 7$	$[6, 2, 13, 5, 8, 3, 10, 11]$
$(3, 6)$	$3, 6$	$[6, 2, 3, 5, 8, 13, 10, 11]$
$(5, 4)$，已交錯	收尾 $1, 4$	$[5, 2, 3, 6, 8, 13, 10, 11]$

回傳 $q = 4$，左側皆 ≤ 6、右側皆 ≥ 6；兩側內部仍不必有序。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同樣五個相等鍵，三種分割有三種結果



字母只辨識資料。雙向例依序交換 $2,5$、$3,4$、$1,3$。對全相等輸入族，雙向版每輪線性掃描、兩側約各半，總時間 $\Theta(n \lg n)$；尾端單向版為 $\Theta(n^2)$，三路為 $\Theta(n)$。這些是指定輸入族的成本；雙向版在其他輸入仍可能退化。

分類一次，可能做了兩次鍵比較

前面的精確式數的是尾端單向掃描中 `key(A[j]) <= x` 的執行次數，每個非 pivot 鍵恰測一次。三路版本若先測 `< x`，失敗才測 `> x`，一次分類可能做兩次布林比較；遇等值鍵時也會測兩次。

例某輪待分類的鍵為 $[1, 4, 3]$、pivot 值 $x = 3$。只看各鍵與 pivot 的分類判斷，鍵 1 用一次測試，鍵 4、3 各用兩次，共五次，而非三次。原地三路版本可能改變處理次序，但每筆的分支測試規則仍要算清楚。

資料結構章節採 0 起算半開區間 $[l, r)$、等值區 $[lt, gt)$；本章三路版本採 1 起算閉區間 $[p, r]$、等值區 $[lt, gt]$。兩者都排除整個等值區，但程式邊界與精確比較常數須各自核對。

鍵互異、pivot 條件均勻時，三路版本的期望時間仍為 $\Theta(n \lg n)$：每輪至多常數倍的線性分類工作，遞迴規模相同。這個漸近結論不表示布林比較數也恰為 $2(n+1)H_n - 4n$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四個秩：比較機率與整次執行的成本不同

四個相異鍵依大小命名為 z_1, z_2, z_3, z_4，輸入排列任意。

鍵的秩對	區間長度	直接比較機率
$(1,2), (2,3), (3,4)$	2	各為 1
$(1,3), (2,4)$	3	各為 $2/3$
$(1,4)$	4	$1/2$

所以 $\mathbb{E}[C] = 3 + 4/3 + 1/2 = 29/6$；代入 $H_4 = 25/12$ 也得到 $2 \cdot 5 \cdot 25/12 - 16 = 29/6$。

單次執行的比較數必為整數。首個 pivot 秩為 2 或 3 時共四次，機率 $1/2$；首選端點、接著三筆中選中位數時共五次，機率 $(1/2)(1/3) = 1/6$；其餘共六次，機率 $1/3$。因此 $4/2 + 5/6 + 6/3 = 29/6$，是對執行成本分布的第二種核對。

相鄰秩必比較，不代表所有鍵對都比較；平均 $29/6$ 也不代表某次真的做了分數次比較。

計數排序：從頻率算出每個鍵的區塊

前提：$K \geq 0$ 為整數，每個鍵都在 $0..K$；輸入、輸出位置為 $1..n$。計數表用鍵值當索引，必須能配置 $K+1$ 格，計數欄位也要能表示 $0..n$。

```
COUNTING-SORT(A,K):
    C[0..K] = 0
    for j = 1 to n: C[key(A[j])] += 1
    for v = 1 to K: C[v] += C[v-1]
    for j = n downto 1:
        v = key(A[j])
        B[C[v]] = A[j]
        C[v] -= 1
    return B
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第一輪掃描後 $C[v]$ 是鍵 v 的頻率；累積後是鍵值 $\leq v$ 的資料數，亦即該鍵區塊的**末位置**。後面的回填會消耗計數；若需要保留區間查詢能力，先複製未消耗的前綴表 P。

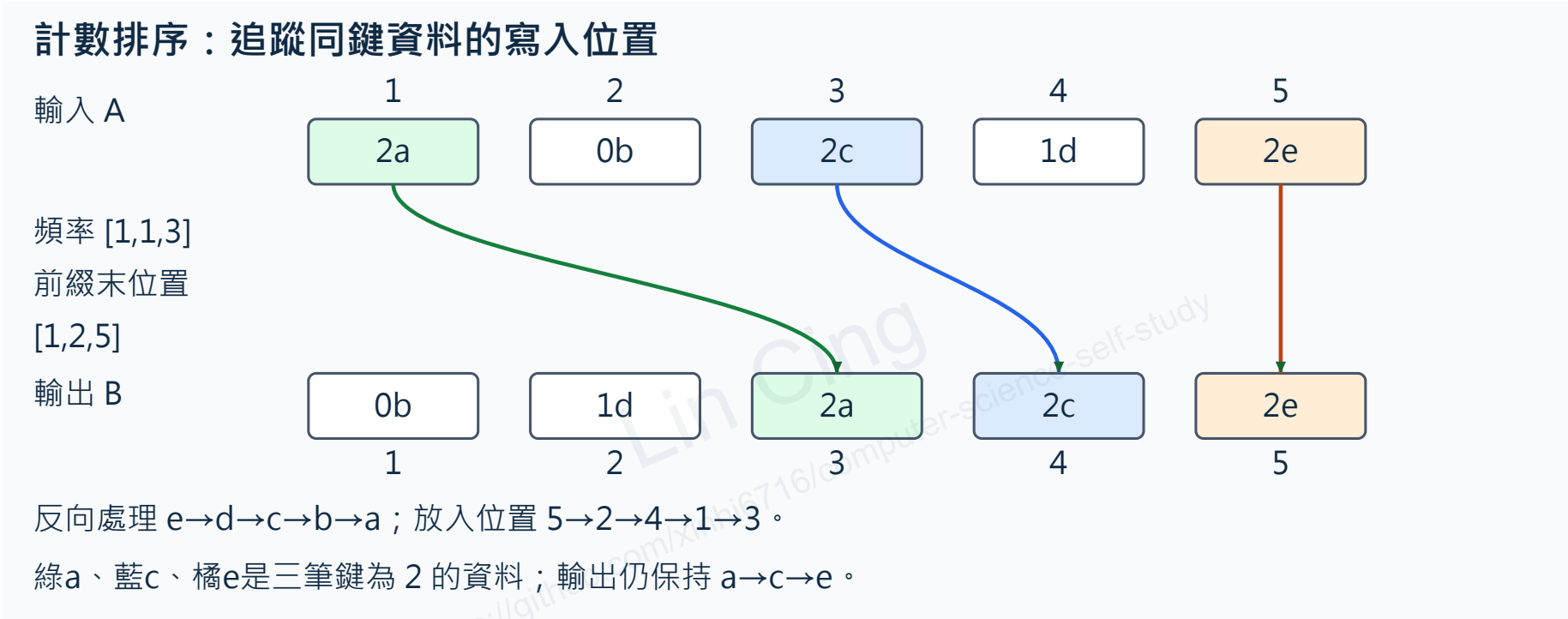
末位置回填：逐筆標出寫入位置

取 $A = [2_a, 0_b, 2_c, 1_d, 2_e]$、$K = 2$。鍵 $0, 1, 2$ 的頻率為 $[1, 1, 3]$；累積末位置為 $[1, 2, 5]$。三個輸出區塊依序是位置 1、位置 2、位置 $3..5$。

掃描位置 j	資料	寫入 B 的位置	寫入後的 C
5	2_e	5	$[1, 2, 4]$
4	1_d	2	$[1, 1, 4]$
3	2_c	4	$[1, 1, 3]$
2	0_b	1	$[0, 1, 3]$
1	2_a	3	$[0, 1, 2]$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同鍵資料的先後順序確實保留下來



輸出 $[0_b, 1_d, 2_a, 2_c, 2_e]$。鍵 2 的身分仍依 a, c, e 出現，這才是穩定性的證據；只看裸鍵 $[0, 1, 2, 2, 2]$ 無法判斷。

回填不變量、穩定性與成本

固定鍵 v，令 $L = P[v-1]$（$v = 0$ 時 $L = 0$）、$R = P[v]$。已處理的輸入後綴中，鍵 v 的資料恰占區塊右端 $[C[v]+1, R]$；$C[v]$ 指向尚未填入的最右位置。

下一筆鍵 v 尚未寫入，所以 $C[v] > L$，寫入合法且不覆蓋既有資料，再減一便保持不變量。終止時 $C[v] = L$，每個區塊恰填滿其頻率大小；區塊依鍵遞增，因此排序正確且資料無增減。

同鍵 x 在輸入先於 y 時，反向掃描先把 y 放到右邊，再把 x 放到左邊，所以穩定。若保留「末位置減一」卻改正向掃描，會反轉同鍵身分；下一節的「起始位置加一」才與正向掃描相配。

四個迴圈分別計初始化、頻率、前綴與輸出，總時間及含輸出的額外空間皆 $\Theta(n + K + 1)$。當 $K = O(n)$ 且 $n \geq 1$ 才是線性；即使只有少數資料，巨大鍵域仍需付表格成本。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

回填前保存前綴計數，才能回答區間查詢

對鍵值 $0 \le a \le b \le K$，先算鍵值不大於 b 的筆數，再扣除小於 a 的筆數：

$$Q(a,b) = \begin{cases} P[b], & a = 0, \\ P[b] - P[a-1], & a > 0. \end{cases}$$

前面五筆資料的三種表格如下：

鍵 v	0	1	2
頻率 $F[v]$	1	1	3
保存的前綴 $P[v]$	1	2	5
回填後的計數器 $C[v]$	0	1	2

$Q(1,2) = 5 - 1 = 4$。若誤用回填後的計數器，會得到錯誤的 $2 - 0 = 2$。另有 $Q(0,0) = P[0] = 1$，實作不必讀取負索引。

必須在回填前複製 P；預處理時間為 $\Theta(n + K + 1)$，每次查詢為 $O(1)$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同樣穩定的計數排序，也能從左往右填

末位置版本從右往左掃描，填入後把計數減一。另一個版本先算每個鍵的**起始位置** $S[v]$，從左往右掃描，填入後加一。

設 $F[v]$ 為鍵 v 的頻率、鍵域 $0..K$：

```
S[0] = 1
for v = 1 to K:
    S[v] = S[v-1] + F[v-1]
for j = 1 to n:
    v = key(A[j])
    B[S[v]] = A[j]
    S[v] = S[v] + 1
```

初始 $S[v] = 1 + \sum_{u<v} F[u]$，是該鍵區塊的第一格。空鍵區塊可以共用同一起始位置，因為不會有資料寫入它。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

起始位置的八筆完整追蹤

取 $A = [3_a, 1_b, 4_c, 2_d, 6_e, 1_f, 2_g, 2_h]$、$K = 6$。鍵 $0..6$ 的頻率為 $[0, 2, 3, 1, 1, 0, 1]$，初始 $S = [1, 1, 3, 6, 7, 8, 8]$。

j	資料	寫入位置	該鍵的下一位置
1	3_a	6	$S[3] = 7$
2	1_b	1	$S[1] = 2$
3	4_c	7	$S[4] = 8$
4	2_d	3	$S[2] = 4$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完成後四筆，核對輸出與最後指標

j	資料	寫入位置	該鍵的下一位置
5	6_e	8	$S[6] = 9$
6	1_f	2	$S[1] = 3$
7	2_g	4	$S[2] = 5$
8	2_h	5	$S[2] = 6$

輸出 $[1_b, 1_f, 2_d, 2_g, 2_h, 3_a, 4_c, 6_e]$。最後指標可等於區塊末端加一；它表示下一個空位，並不表示這一格還會被存取。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同鍵資料沿輸入順序填入同一區塊



1_b 先占位置 1，1_f 再占位置 2；$2_d, 2_g, 2_h$ 依序占位置 3、4、5。顏色連結同鍵區塊，身分標籤用來核對穩定性。

已填前綴與下一空位的不變量

令 $S_0[v]$ 是未消耗的起始位置，$c_j(v)$ 是已掃描的前 $j-1$ 筆中、鍵為 v 的資料數。每輪開始時，

$$S[v] = S_0[v] + c_j(v).$$

區塊 $[S_0[v], S[v]-1]$ 恰好裝著已掃描的鍵 v 資料，而且順序與輸入相同；這些區塊互不重疊。

初始化 $c_1(v) = 0$，所有區塊為空。讀到鍵 v 時，尚有這筆未放，所以 $c_j(v) < F[v]$，寫入位置仍在 v 的合法區塊內。只寫下一空位再令 $S[v]$ 加一，保持資料與次序；其他鍵的區塊不變。

終止時 $c_{n+1}(v) = F[v]$，每個區塊恰填滿，區塊依鍵排序，故正確且穩定。掃描、計數及起始位置各自都是完整迴圈，時間 $\Theta(n+K+1)$、含輸出的額外空間 $\Theta(n+K+1)$。

掃描方向與位置定義要成對使用： 正向配起始位置加一；反向配末位置減一。只改掃描方向，可能仍排對裸鍵，卻反轉同鍵資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LSD 基數排序：穩定性把低位順序保留下來

設每個鍵是固定 $d \geq 1$ 位、基底 $r \geq 2$ 的非負整數；不足位數補前導零。LSD（最低有效位優先）依序處理第 $1, 2, \ldots, d$ 位，每輪對目前位做**穩定**排序。

不變量：完成第 t 輪後，序列依最低 t 位代表的數值排序。$t = 0$ 無限制。第 t 輪把資料按目前位分組，不同組由該位決定大小；同組內靠穩定性保留前 $t - 1$ 位的次序。因此最低 t 位完整有序，$t = d$ 時整體有序。

保留低位順序只發生在**相同高位組內**，不是全域仍依低位遞增。反例：$[12, 11]$ 先按個位得到 $[11, 12]$；十位皆為 1，若不穩定地反轉同鍵，便錯誤地回到 $[12, 11]$。

直接對整個序列按百位、十位、個位依次排序，也會把低位變成最後的主序。MSD 必須在同一高位群內遞迴處理，不能只把 LSD 的迴圈倒過來。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三輪基數排序，每輪都列出完整序列



百位輪中，205 排在 212 前面，是因為同一百位組內保留了低兩位 $05 < 12$ 的次序。每輪都只比較目前位，完整鍵序由不變量逐步建立。

逐輪核對基數排序的完整結果

已處理的位	完整序列
尚未處理	$[412, 205, 421, 115, 212, 405]$
個位	$[421, 412, 212, 205, 115, 405]$
個位與十位	$[205, 405, 412, 212, 115, 421]$
全部三位	$[115, 205, 212, 405, 412, 421]$

十位輪中，$205, 405$ 的十位皆為 0，保留前輪的相對次序。$412, 212, 115$ 的十位皆為 1，也保留此前次序；它們的最低兩位依序為 $12, 12, 15$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

位元數、每輪基底與成本一起算

每輪用穩定計數排序，時間 $\Theta(n+r)$；d 輪共 $\Theta(d(n+r))$，可重用的額外空間 $\Theta(n+r)$。只有 d 固定且 $r = O(n)$ 才直接得到線性時間。

對 $b \geq 1$ 位元鍵，每輪取 s 位，$1 \leq s \leq b$，則

$$r = 2^s, \qquad d = \lceil b/s \rceil, \qquad T = \Theta(\lceil b/s \rceil (n + 2^s)) \,.$$

最高一組不足 s 位時可補零。假設一個鍵與一個數位可在常數時間讀取；超過機器字長的大整數，必須另外計取位與搬移成本。

$n \geq 2$ 時取 $s = \min(b, \lfloor \lg n \rfloor)$，便有 $2^s \leq n$，得到 $O(n\lceil b/s \rceil)$；$b = O(\lg n)$ 時是 $O(n)$。

在「每輪成本為 $n + r$」的模型下，32 位元鍵取 $s = 8$ 需 $4(n + 256)$，取 $s = 16$ 需 $2(n + 65536)$。後者較小當且僅當 $n > 65024$；等號時相同。這是指定模型的比較，實機還有配置、快取與頻寬成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分桶是共同技巧，並不代表同一個演算法

方法	分配依據	分配後還做什麼
LSD radix sort	由低位到高位，每次一位	每輪對整個序列穩定排序
MSD radix sort	由最高位開始	只在同一高位群內遞迴處理下一位
區間 bucket sort	$x \in [0, 1)$ 映到 $\lfloor nx \rfloor$	每桶完成內部排序，再按桶串接

三者都可以把資料放進 buckets，但**位數、遞迴規則與成本前提不同**。固定十個數位桶，不等於依 n 分成 n 個等長區間；把每桶直接排序，也不等於繼續按下一位遞迴。

若把 $[312, 305, 121, 118, 214, 207]$ 按百位分組，得到 $[121, 118]$、$[214, 207]$、$[312, 305]$。這一步尚未完成排序：MSD 要在各組處理低位；區間桶方法則須用指定的桶內排序，把每組排好。只串接原分組會得到錯誤次序。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

區間桶排序：先分配，再把每桶排好

輸入 $n \geq 1$ 個 $x \in [0, 1)$；建立 n 個桶 $B[0..n-1]$，將 x 依序附加至 $B[\lfloor nx \rfloor]$。每桶用插入排序，最後依桶編號串接。空輸入直接返回；$x = 1$ 不在定義域，不能拿來存取 $B[n]$。

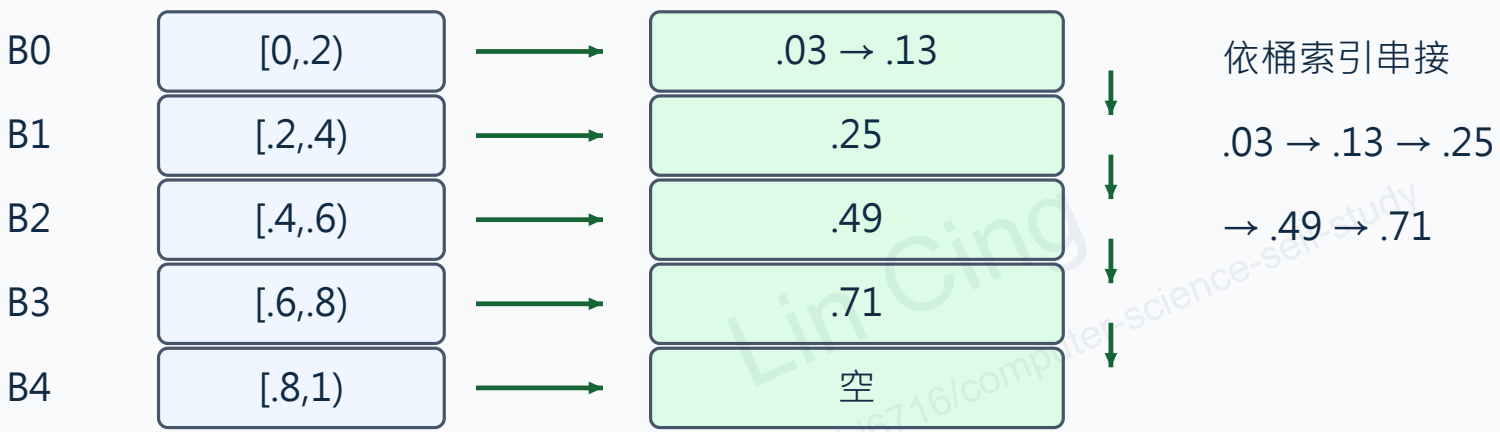
正確性不需要隨機分布：若 $x \leq y$，則 $\lfloor nx \rfloor \leq \lfloor ny \rfloor$。不同桶已依不重疊的半開區間排序，同桶則由內部排序完成。依輸入順序附加，再用穩定插入排序，才能另保證穩定性；頭插可能先改變同鍵順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

五個桶：區間、桶內次序與完整輸出

桶排序：區間先決定不同桶的次序

輸入 .13,.25,.03,.71,.49 → 索引 0,1,0,3,2



藍：桶的半開區間；綠：桶內排序結果；跨桶無需再比較。

例 $n = 5$，輸入 $[.13, .25, .03, .71, .49]$，桶編號依序 $[0, 1, 0, 3, 2]$。第 0 桶排成 $[.03, .13]$，第 $1, 2, 3$ 桶分別為 $.25, .49, .71$，第 4 桶為空；輸出 $[.03, .13, .25, .49, .71]$。

期望線性：算平方桶大小，而非平方期望

令 N_i 是桶 i 的大小。建立、分配與串接共 $\Theta(n)$，桶內插入排序總成本至多常數倍的 $\sum_i N_i^2$。因此

$$T \le c_0 n + c_1 \sum_{i=0}^{n-1} N_i^2.$$

現在才加入輸入假設：各筆鍵**互相獨立且均勻**落在 $[0,1)$。令 X_{ij} 表示第 j 筆落入桶 i，則 $N_i = \sum_j X_{ij}$、$X_{ij}^2 = X_{ij}$，且

$$\begin{aligned}\mathbb{E}[N_i^2] &= \sum_j \mathbb{E}[X_{ij}] + \sum_{j \ne k} \mathbb{E}[X_{ij} X_{ik}] \\ &= n\frac{1}{n} + n(n-1)\frac{1}{n^2} = 2 - \frac{1}{n}.\end{aligned}$$

所以 $\mathbb{E}[\sum_i N_i^2] = 2n - 1$，給 $\mathbb{E}[T] = O(n)$；讀取輸入本身給 $\Omega(n)$，合為 $\Theta(n)$。

獨立性用在不同資料 j, k 的落桶事件。不同桶的大小 N_i 彼此相關，因為總和固定為 n；這不妨礙期望線性性。僅知 $\mathbb{E}[N_i] = 1$，不能把 $\mathbb{E}[N_i^2]$ 寫成 1。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

分布集中、資料相關與真正的最壞輸入

若各筆仍獨立，但落入桶 i 的機率改為 p_i，$\sum_i p_i = 1$，同樣展開得到

$$\mathbb{E}\left[\sum_i N_i^2\right] = n + n(n-1)\sum_i p_i^2.$$

當 $\sum_i p_i^2 = O(1/n)$ 時仍有線性期望上界。均勻分布給 $1/n$；若所有機率集中單桶，平方桶大小總和就是 n^2。

邊際均勻也不夠：先均勻抽 $J \in \{0, \dots, n-1\}$，令每筆 $x_j = (J + 1/2)/n$。各筆對每個桶的邊際機率都為 $1/n$，卻總是全體同桶，$\sum_i N_i^2 = n^2$。這反駁平方成本的錯誤估計；相等鍵的插入排序本身仍可能只需線性時間。

真正達到平方時間的輸入族可取 $n \ge 2$、$x_j = (n - j + 1)/(2n^2)$。它們互異、嚴格遞減，而且全在第 0 桶；插入排序須搬移 $n(n-1)/2$ 次，故最壞時間 $\Theta(n^2)$。

若每桶改用最壞 $O(m \lg(m+1))$ 的排序，總最壞可降為 $O(n \lg(n+1))$；獨立均勻模型仍保留線性期望，因為 $m \lg(m+1) = O(m^2)$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

次序統計量與極值：先說清楚要找哪一筆

輸入為 $n \geq 1$ 筆可全序比較的鍵；第 k 小是排序後多重集合的第 k 個值，$1 \leq k \leq n$。重複值各占一個位置，答案卻可相同。例如 $[2, 5, 2, 8]$ 的第 $1, 2$ 小都是 2。

本章的下中位數（lower median）取 $k = \lceil n/2 \rceil$。偶數筆時，它是兩個中央值中較小的值，與統計上取兩者平均的定義不同。空輸入與非法秩在入口拒絕。

找最小值可從第一筆開始，依序比較其餘資料；不變量是「目前最小值等於已讀前綴的最小值」，共做 $n - 1$ 次鍵比較。最大值同理。

這也是相異鍵輸入的比較下界：每個非最小值都須至少在一次比較中被證明大於另一個鍵，才能排除最小值資格；一次比較至多排除一個尚未被排除的候選。因此至少 $n - 1$ 次。用相異鍵證明最壞下界，並不要求演算法拒絕重複鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成對找極值：八筆資料只做十次比較

同時找最小與最大時，先比較一對資料：較小者只和目前最小值比較，較大者只和目前最大值比較。每對共三次，比兩筆分別挑戰兩個極值少一次。

取 $[14, 3, 9, 1, 8, 5, 12, 6]$。偶數筆先比較第一對以初始化；後面各對維持「兩個極值涵蓋已處理前綴」的不變量。

本輪資料	組內較小、較大	更新後最小值	更新後最大值	累積比較次數
$14, 3$	$3, 14$	3	14	1
$9, 1$	$1, 9$	1	14	4
$8, 5$	$5, 8$	1	14	7
$12, 6$	$6, 12$	1	14	10

偶數 n 用 $1 + 3(n-2)/2 = 3n/2 - 2$ 次；奇數 n 先以第一筆同時初始化兩個極值，剩下各對共 $3(n-1)/2$ 次。統一為 $\lceil 3n/2 \rceil - 2$；$n = 1$ 時為零。平手時仍可任選一方作極值，數值答案正確。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

為什麼成對法的最壞比較數已經最少

對相異鍵，每筆最初同時具有最小值、最大值兩種候選資格。最後只保留兩種資格各一個，故總共須排除 $2n - 2$ 個資格。

對手（adversary）可依下表回答比較。稱未曾比較者為 U，只剩最大值資格者為 H，只剩最小值資格者為 L，兩者都失去者為 M。

比較對象	對手回答與新排除數
U, U	一大一小，各失一種資格；排除 2 個
H, H 或 L, L	消去一個最大或最小候選；排除 1 個
H, L	回答 $H > L$；排除 0 個
U 與已分類者	若對方是 H，令 $U < H$；是 L，令 $U > L$；是 M，任選一致方向。只排除 U 的一種資格
涉及 M、不涉及 U	保持 $H > M > L$；M, M 選既有偏序的一個線性延伸，至多排除 1 個

Lin Cing https://github.com/xinhi6716/computer-science-self-study

把候選資格的排除次數算清楚

這些回答維持一致全序：H 從未被證明小於其他鍵，L 從未被證明大於其他鍵，U 沒有比較約束。按 $H > M > L$ 分層，層內採既有比較偏序的線性延伸，不會產生循環。

只有 U, U 比較能一次排除兩個資格，且每次耗掉兩筆未比較資料，至多 $\lfloor n/2 \rfloor$ 次。其他比較至多排除一個資格。若總比較數為 c，則

$$2n - 2 \le c + \lfloor n/2 \rfloor, \qquad c \ge \lceil 3n/2 \rceil - 2.$$

比較前

16 個候選資格

8 × (min + max)

−8 →

四次組內比較後

8 個候選資格

H: 14, 9, 8, 12

L: 3, 1, 5, 6

−6 →

再做六次極值比較後

2 個候選資格

max: 14; min: 1

先集中組內比較：4 × 2 + 6 × 1 = 14 個資格；4 + 3 + 3 = 10 次比較。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

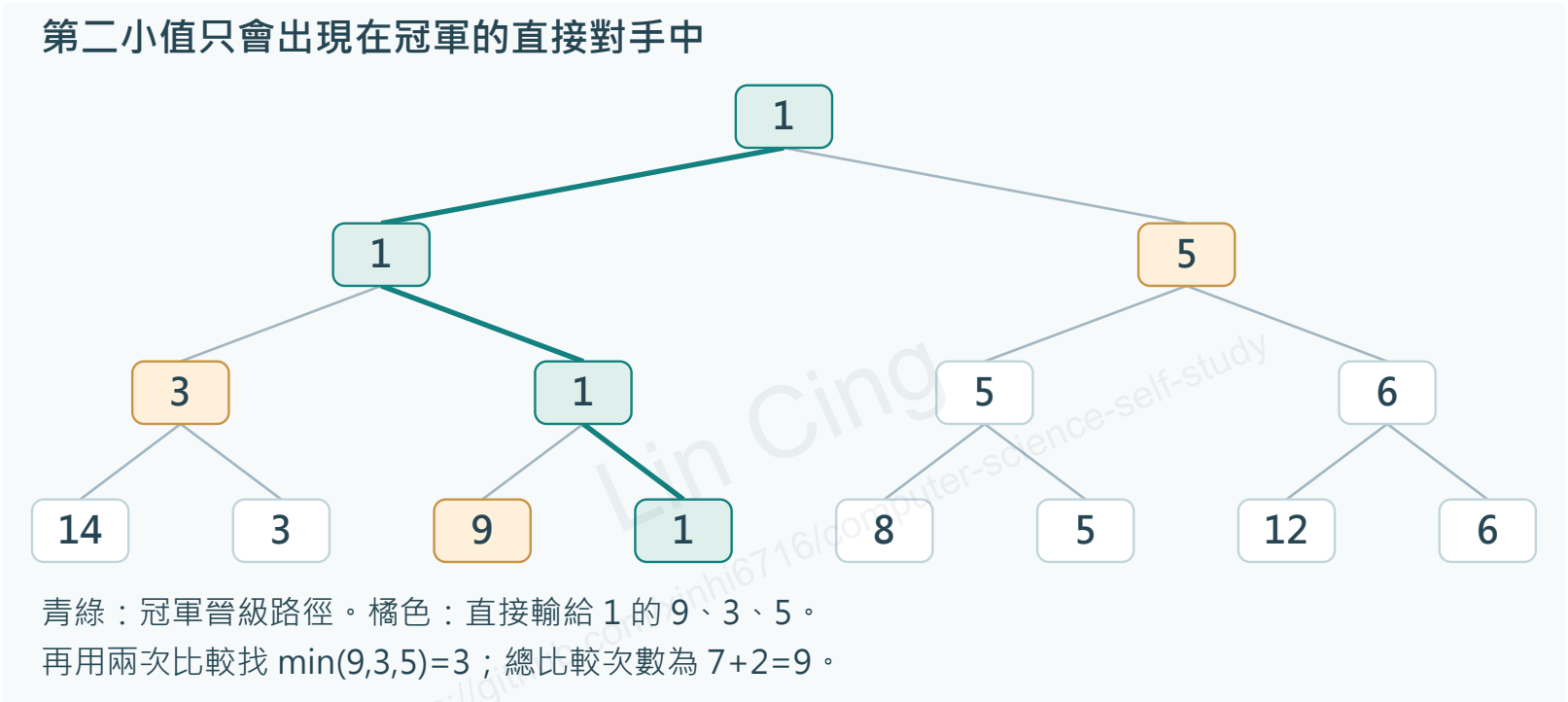
淘汰賽保存了第二小值的候選名單

讓較小鍵晉級，並保存每位勝者直接擊敗的對手。第二小值必須直接輸給最小值：若它輸給另一個非最小鍵，已有兩個鍵比它小，產生矛盾。以下先假設鍵相異。

一般 $n \geq 2$ 可用平衡賽程，不足的席位輪空，無須與虛構鍵比較。每次淘汰一人，共 $n-1$ 次；冠軍實際對手數 $d \leq \lceil \lg n \rceil$，清單再比 $d-1$ 次，最壞至多 $n + \lceil \lg n \rceil - 2$。若允許重複鍵，可按 $(key, identity)$ 打破平手，此時求的是第二筆資料，未必是第二個不同值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

八鍵淘汰賽：冠軍直接擊敗了誰



冠軍 1 直接擊敗 $9, 3, 5$，故第二小是這三者的最小值 3。淘汰賽共 7 次，候選清單再比 2 次，共 9 次。

隨機選擇的三路分割：把等值區一次排除

排序會遞迴處理兩側；選擇只保留含答案的一側。本節陣列索引是 **1 起算閉區間** $A[p..r]$；目標 k 是此子陣列內的局部秩，$1 \le k \le r - p + 1$。分割比較鍵值、交換整筆資料，x 為子陣列中實際出現的鍵。

```
PARTITION3(A,p,r,x):
    lt = p; i = p; gt = r
    while i <= gt:
        if key(A[i]) < x:
            swap(A[lt],A[i]); lt += 1; i += 1
        elif key(A[i]) > x:
            swap(A[i],A[gt]); gt -= 1
        else:
            i += 1
    return (lt,gt)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三路分割：四區不變量與每輪進度

迴圈開始時，$A[p..lt-1] < x$、$A[lt..i-1] = x$、$A[i..gt]$ 未知、$A[gt+1..r] > x$；空區間合法。小鍵分支擴大小區與等值區；等鍵分支擴大等值區。大鍵分支從右端換來的鍵尚未分類，所以不能推進 i。每輪未知區長度減一，交換保留多重集合；終止時未知區消失，得到完整三區。

< x　= x　尚未分類　> x　pivot x = 3; A[4..10]

lt = 4, i = 6, gt = 9

4	5	6	7	8	9	10
3	3	5	3	9	1	8

lt = 4, i = 6, gt = 8

4	5	6	7	8	9	10
3	3	1	3	9	5	8

交換 A[6] 的 5 與 A[9] 的 1：gt 減成 8，i 仍為 6。

下一輪必須分類新換來的 A[6] = 1；p = lt = 4，故小於 x 的區間目前為空。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

隨機選擇：只有往右才要換算局部秩

```
RANDOM-SELECT(A,p,r,k):
    require 1 <= k <= r-p+1
    while p < r:
        x = key(A[uniform-integer(p,r)])
        (lt,gt) = PARTITION3(A,p,r,x)
        L = lt-p; E = gt-lt+1
        if k <= L: r = lt-1
        elif k <= L+E: return x
        else: k -= L+E; p = gt+1
    return key(A[p])
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

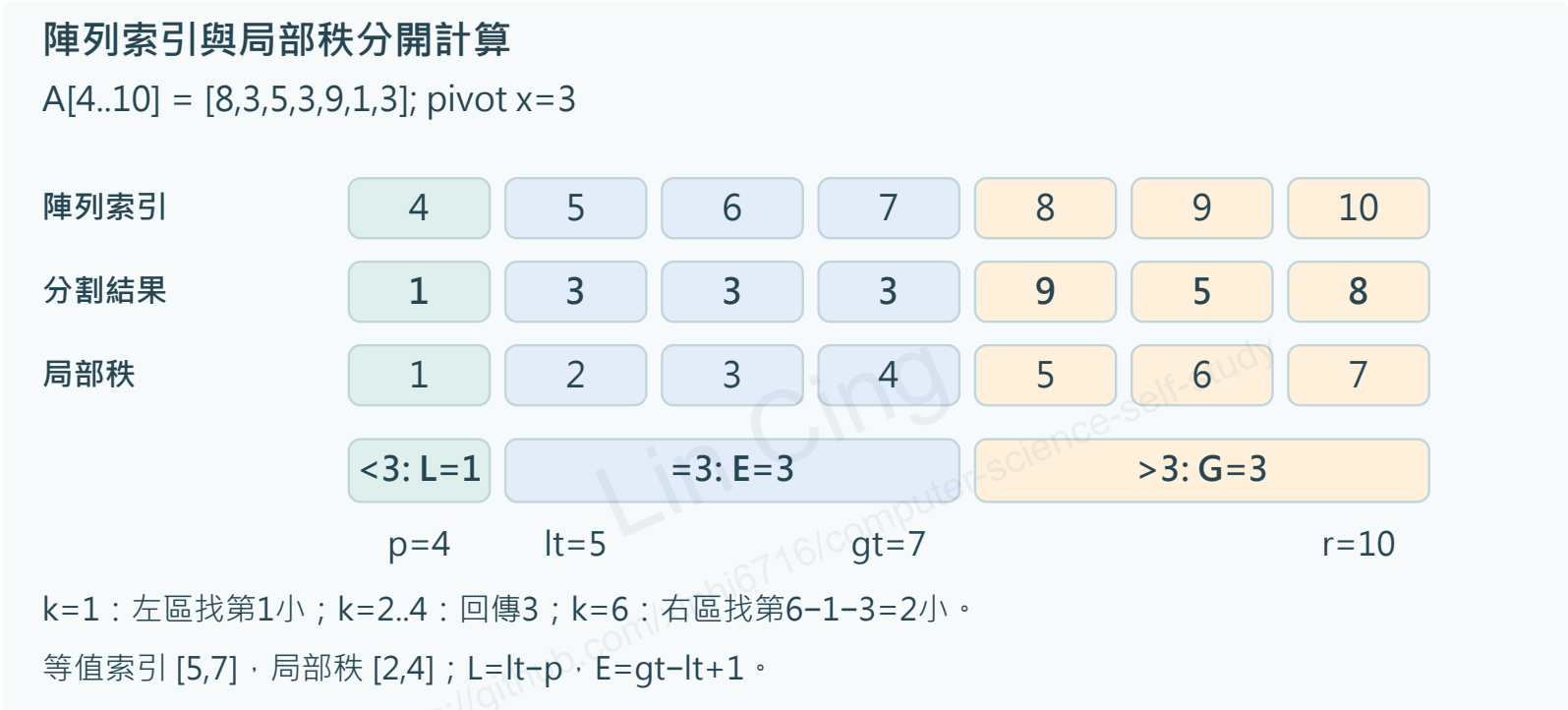
選擇迴圈：答案不變，規模持續縮小

x 取自目前資料，因此 $E \geq 1$。答案落在局部秩 $L+1..L+E$ 時直接回傳 x；往左保留第 k 小，往右則扣除已確定較小的 L 筆與等值的 E 筆。

迴圈不變量：目前子陣列的第 k 小，等於原問題要求的鍵值。左分支滿足 $1 \leq k \leq L$；右分支滿足 $1 \leq k-L-E \leq r-gt$，故不會保留空區間。每次未返回時，規模至少減少 E；最後只剩一筆，回傳該鍵。這同時證明正確性與終止，允許重複鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

局部秩、區長與絕對索引要分開



$A[4..10] = [8, 3, 5, 3, 9, 1, 3]$，取 $x = 3$，分割後 $lt = 5, gt = 7$，所以 $L = 1, E = 3, G = 3$。局部等值秩為 $2..4$。

用區長逐一換算目標局部秩

原局部秩	判斷	下一狀態或答案
1	$1 \le L$	$A[4..4]$ 的第 1 小
4	$L < 4 \le L + E$	回傳 3
6	$6 > L + E$	$A[8..10]$ 的第 $6 - 1 - 3 = 2$ 小

不能直接比較 k 與 lt：前者由目前子陣列的第一筆算起，後者從整個陣列的索引起點算起。若改用絕對目標位置 $t = p + k - 1$，則每輪保留 t，不再同時扣局部秩。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機選擇：對所有後續狀態做強歸納

條件於目前的陣列，pivot 均勻選自其 n 筆資料；未來每輪亦在當時的子陣列內重新均勻選取。先假設鍵相異，pivot 的局部秩 R 均勻分布於 $1..n$。無論目標秩為何，後續規模至多

$$m_R = \max(R-1,\, n-R).$$

令 $F(A,k)$ 為此狀態的剩餘期望成本。本層掃描與固定開銷至多 cn。對**所有較短狀態**假設 $F(B,j) \le a|B|$，條件於本次 pivot 套用歸納，得

$$F(A,k) \le cn + \frac{a}{n}\sum_{r=1}^{n} \max(r-1,\, n-r).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把所有較大側長度加總，再完成歸納

偶數 $n = 2h$ 時，較大側長度 $h, h+1, \dots, 2h-1$ 各出現兩次，故和為 $2\sum_{t=h}^{2h-1} t = 3h^2 - h$。奇數 $n = 2h+1$ 時，中央秩給 h，其餘對稱出現兩次，和為 $h + 2\sum_{t=h+1}^{2h} t = 3h^2 + 2h$。除以 n 後均至多 $3n/4$，因此

$$F(A,k) \le (c + 3a/4)n \le an \qquad (a \ge 4c).$$

增大 a 以涵蓋 $n = 1$ 基例，即得 $O(n)$。首次分割須掃描 $n > 1$ 筆，故期望為 $\Theta(n)$。

證明沒有假設未知的成本函數單調，也沒有把相依隨機變數的期望拆乘。$3n/4$ 是較大側長度的期望上界，不能當成每次分割的確定上限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

重複鍵不破壞三路選擇的期望線性界

為分析替同鍵資料附上固定的身分次序，得到唯一秩 $R \in \{1, \dots, n\}$；實際分割仍只比較鍵，不比較身分。均勻選一筆資料，使分析用的 R 均勻。

若答案在 $< x$ 區，其長度至多 $R - 1$；若在 $> x$ 區，其長度至多 $n - R$；若在等值區便終止。故每種 pivot 身分都有同一上界 $m_R = \max(R - 1, n - R)$，前述對所有較短狀態的強歸納照用，得到期望 $\Theta(n)$。

最壞執行仍可為 $\Theta(n^2)$：相異鍵、找最小值，每輪恰抽到當前最大值，掃描規模為 $n, n-1, \dots, 2$。這條路徑有正機率，期望界不等於最壞界。迭代版加原地三路分割只需 $O(1)$ 額外空間，且會改動輸入。

全等資料則一次掃描就返回。若改用把所有等值都送到同一側的二路 Lomuto，找最小秩時會退化，隨機 pivot 也無法補救。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

確定性選擇：讓完整程序與追蹤使用同一規格

以下列表與排序結果皆為 **1 起算**。SORT-COPY 依鍵排序副本；STABLE-SPLIT-COPY 線性掃描輸入，依序加入小於、等於、大於 x 的三個新列表，保持各區的輸入次序。這是便於自學追蹤的 $O(n)$ 額外空間版本；不把它宣稱為原地程序。

每組至多五筆，取下中位數；組數 $g = \lceil n/5 \rceil$，再對組中位數遞迴找第 $\lceil g/2 \rceil$ 小，作為 pivot。

```
MEDIAN-PIVOT(A):
    M = empty list
    for each consecutive group G of at most 5 records in A:
        B = SORT-COPY(G)
        append key(B[ceil(length(B)/2)]) to M
    return DETERMINISTIC-SELECT(M,ceil(length(M)/2))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

確定性選擇：回傳等值或遞迴單側

```
DETERMINISTIC-SELECT(A,k):
    n = length(A); require 1 <= k <= n
    if n <= 5: return key(SORT-COPY(A)[k])
    x = MEDIAN-PIVOT(A)
    (L,E,G) = STABLE-SPLIT-COPY(A,x)
    if k <= length(L):
        return DETERMINISTIC-SELECT(L,k)
    elif k <= length(L)+length(E): return x
    else:
        return DETERMINISTIC-SELECT(G,k-length(L)-length(E))
```

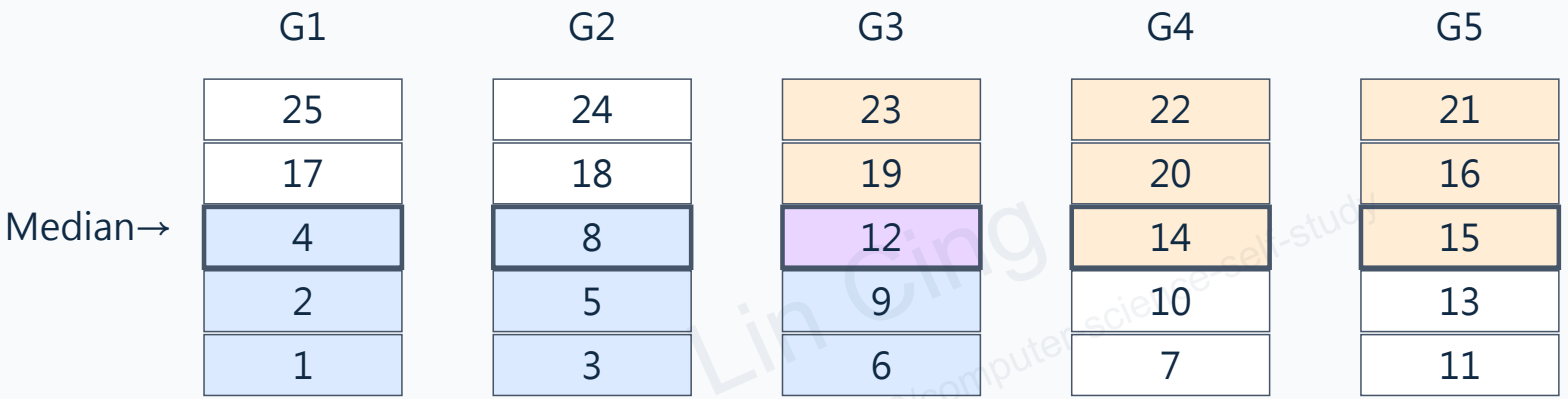
正確性由分割的三區次序與規模歸納得到。組中位數的遞迴規模 $g < n$；答案側至多 $n - 1$，因 x 必出現，等值區非空。兩種遞迴都會停止，時間分析也必須同時計入兩者。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

五元素組提供的是排除證據，不是精確中位數

Median-of-medians：每個完整組提供三筆證據

每欄一組，越上方越大；中間一列是組Median。Pivot x=12。



組Median [4,8,12,14,15] → 選第3小12；全體真正Median為13。

藍+紫：≤12共9筆　橘+紫：≥12共9筆　紫：共有Pivot

這 25 筆資料恰為 1..25。組中位數為 $[4, 8, 12, 14, 15]$，所以 $x = 12$；全體下中位數卻是第 13 小，也就是 13。

兩側證據，都列出實際鍵值

中位數 ≤ 12 的前三組，各取其最小三筆，可列出 $[1, 2, 4]$、$[3, 5, 8]$、$[6, 9, 12]$，共九筆 ≤ 12。中位數 ≥ 12 的後三組，各取最大三筆，可列出 $[12, 19, 23]$、$[14, 20, 22]$、$[15, 16, 21]$，共九筆 ≥ 12。

兩份證據在 12 重疊沒有問題：若遞迴找嚴格大側，排除的是全部 $\leq x$；若找嚴格小側，排除的是全部 $\geq x$。分析的兩個分支不會同時採用。含重複鍵時仍保證非嚴格的大小關係，三路分割正好把等值區一併排除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

取整與殘組：完成最壞線性時間的歸納

$g = \lceil n/5 \rceil$ 個組中位數的下中位數 x，兩側各至少有 $\lceil g/2 \rceil$ 個中位數滿足 $\le x$ 或 $\ge x$。保守扣掉可能不足五筆的殘組，以及含所選 pivot 身分的那一組，剩至少 $\lceil g/2 \rceil - 2$ 個完整組。

每組提供三筆證據，所以兩側各至少有

$$3(\lceil g/2 \rceil - 2) \ge 3n/10 - 6$$

筆非嚴格大小證據，嚴格遞迴側至多 $7n/10 + 6$。小規模下這個下界可能為負，只代表界太鬆；實際等值區非空，子問題仍嚴格縮小。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

歸納同時付清兩個遞迴的成本

非遞迴工作至多 cn。對所有較短狀態假設成本至多常數 a 乘其長度；$n \geq 140$ 時，

$$\begin{aligned} T(n) &\leq a\lceil n/5 \rceil + a(7n/10 + 6) + cn \\ &\leq (9a/10 + c)n + 7a \leq an. \end{aligned}$$

最後一步取 $a \geq 20c$，使 $a/10 - c \geq a/20$，且 $n \geq 140$ 給 $an/20 \geq 7a$。有限的 $1 \leq n < 140$ 可增大 a 涵蓋；這是證明門檻，不要求改動程式的五筆基例。初次分組掃描給 $\Omega(n)$，故最壞時間為 $\Theta(n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第十六小：逐次列出分組、區長與下一個秩

輸入按以下五組串接，分組前不先排序全體：

$[1,2,4,17,25]$、$[3,5,8,18,24]$、$[6,9,12,19,23]$、$[7,10,14,20,22]$、$[11,13,15,16,21]$。

目前 (n,k)	組中位數 M 與 pivot x	$(\lvert L\rvert,\lvert E\rvert,\lvert G\rvert)$	下一步
$(25,16)$	$[4,8,12,14,15]\to 12$	$(11,1,13)$	大側第 4 小
$(13,4)$	$[19,20,16]\to 19$	$(6,1,6)$	小側第 4 小
$(6,4)$	$[15,16]\to 15$	$(2,1,3)$	大側第 1 小
$(3,1)$	基例排序 $[16,17,18]$	不再分割	回傳 16

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

由保留的列表，重建下一次分組

第一次大側保留掃描次序，為 $[17, 25, 18, 24, 19, 23, 14, 20, 22, 13, 15, 16, 21]$。分成五、五、三筆，排序副本後分別是 $[17, 18, 19, 24, 25]$、$[13, 14, 20, 22, 23]$、$[15, 16, 21]$，中位數為 $19, 20, 16$；在這三筆上用基例回傳中位數 19。

接下來小側為 $[17, 18, 14, 13, 15, 16]$；前五筆的中位數為 15，單筆殘組為 16，兩筆取下中位數 15。大側為 $[17, 18, 16]$，第 1 小回傳 16。這條路徑符合前頁的穩定列表程序；改成原地交換分割，區內次序及後續分組可能不同，不能要求同一條中間路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

改變組大小，時間與空間保證都要重算

固定奇數組大小 $s = 2t + 1$，每個完整組的中位數可提供 $t + 1$ 筆非嚴格證據。忽略取整與常數項時，兩個遞迴規模比例和為

$$\frac{1}{s} + 1 - \frac{t+1}{2s} = 1 + \frac{1-t}{2s}.$$

五筆一組時為 $9/10 < 1$，七筆一組時為 $6/7 < 1$，留有線性工作所需的餘裕；有限殘組與歸納基例仍須另算。三筆一組時比例和為 1，粗界為

$$T(n) \le T(\lceil n/3 \rceil) + T(2n/3 + O(1)) + O(n).$$

此式可證 $O(n \lg n)$，卻不能只憑一個上界便聲稱實際演算法必有 $\Omega(n \lg n)$。下界還須證明相應的壞分割能在各層持續發生。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

副本與尚未返回的呼叫鏈，都要計入空間

本章穩定列表程序的額外空間為 $O(n)$：每次呼叫保存 $O(n)$ 筆副本，兩個遞迴先後執行；對充分大規模，每條尚未返回的呼叫鏈至多按固定小於一的比例縮小，幾何和為 $O(n)$，小規模部分為常數。最壞呼叫深度為 $O(\lg n)$。把分割換成原地版本還不等於整體 $O(1)$ 空間，因為組中位數、副本與遞迴堆疊都必須處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

生日碰撞：用補事件，逐人算不碰撞的機率

模型：$k \geq 0$ 個人各自獨立、均勻選擇 $n \geq 1$ 種日期。這是簡化的機率模型，不直接假設真實生日符合它。

當前 j 人日期都不同，下一人有 $n - j$ 種日期不會碰撞，所以條件機率為 $(n - j)/n$。由條件機率連乘，$k \leq n$ 時

$$\Pr(\text{日期皆不同}) = \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right),$$

$$\Pr(\text{至少一對同日}) = 1 - \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right).$$

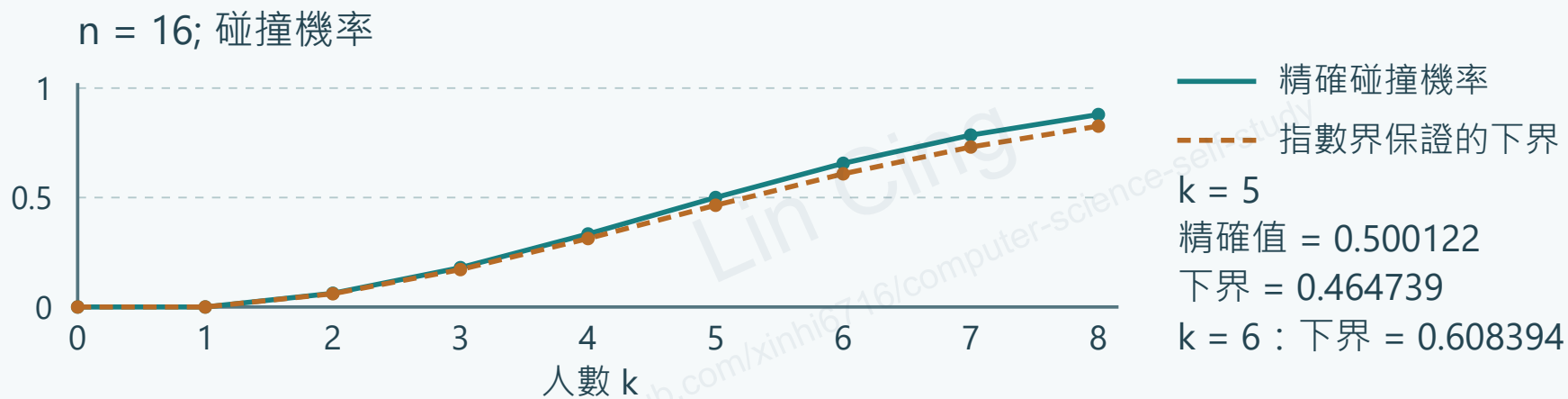
$k = 0, 1$ 無碰撞；空乘積取 1。$k > n$ 時由鴿籠原理必碰撞，直接取機率 1，不要把連乘繼續延伸成負因子。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

充分條件可能比精確門檻保守

對 $2 \le k \le n$，逐項使用 $1 - x \le e^{-x}$，再取補事件：

$$\Pr(\text{碰撞}) \ge 1 - \exp\left(-\sum_{j=0}^{k-1} \frac{j}{n}\right) = 1 - e^{-k(k-1)/(2n)}.$$



下界達到 $1/2$ 的條件是 $k(k-1) \ge 2n \ln 2$；這是**充分條件**，並非精確門檻。$n = 16$ 時，精確機率在 $k = 5$ 已超過 $1/2$，這個下界要到 $k = 6$ 才能保證。

碰撞對數的期望，不等於發生碰撞的機率

令 I_{ij} 表示第 i, j 人同日。兩人獨立均勻，故

$$\Pr(I_{ij} = 1) = \sum_{d=1}^{n} \Pr(B_i = d, B_j = d) = n\frac{1}{n^2} = \frac{1}{n}.$$

碰撞對數 $X = \sum_{i<j} I_{ij}$，所以

$$\mathbb{E}[X] = \frac{\binom{k}{2}}{n}.$$

「有碰撞」是事件 $X \geq 1$。Markov 界只給 $\Pr(X \geq 1) \leq \mathbb{E}[X]$，不能反向推論：期望很大，不代表非零機率很大。例如 X 以 $1/M$ 機率等於 M^2，否則為零，則期望為 M，非零機率卻為 $1/M$。

在生日的獨立均勻模型中，$k = o(\sqrt{n})$ 時由上界得碰撞率趨零；$k \geq C\sqrt{n}$、固定 C 夠大時，由補事件指數界得到固定正的碰撞率。兩個方向合起來，才支持 $\sqrt{n}$ 的機率轉折尺度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

十六種日期：把每一個條件機率乘出來

取 $n = 16, k = 5$，逐人累乘不碰撞的機率：

加入第幾人	本次不碰撞的條件機率	到此仍皆不同的機率
1	1	1
2	$15/16$	$15/16$
3	$14/16$	$105/128$
4	$13/16$	$1365/2048$
5	$12/16$	$4095/8192$

碰撞機率為 $4097/8192 \approx 0.500122$，碰撞對數的期望則為 $\binom{5}{2}/16 = 5/8$。$k = 5$ 的指數補事件上界為 $e^{-10/16} > 1/2$，尚不能證明碰撞超過半數；$k = 6$ 時才有 $e^{-15/16} < 1/2$。充分條件較保守，與精確答案沒有矛盾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一組樣本的碰撞，與整個模型的平均不同

同一日期樣本與整個機率模型：樣本計數和期望要分開



模型：16日，5人。P(有碰撞)=4097/8192≈0.500122。

E[碰撞對數]=10/16=5/8；這個期望不是上面的事件機率。

圖中的日期為 $[2, 7, 10, 7, 4]$，本次只有第 $2, 4$ 人同日，因此 $X = 1$、$I_{\{X \geq 1\}} = 1$。它們是這一個結果的計數，與對全部結果平均的 $5/8$、$4097/8192$ 不同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

兩兩獨立足以算單對，卻不足以連乘全體

令 U, V 獨立均勻於 $\{0, 1, 2\}$，三人的日期為

$$B_1 = U, \qquad B_2 = V, \qquad B_3 = (2U - V) \bmod 3.$$

任取兩個日期，皆能唯一還原 (U, V)：B_1, B_2 直接還原；B_1, B_3 給 $V = 2B_1 - B_3$；B_2, B_3 給 $U = 2(B_2 + B_3)$，均在模 3 下計算。因此每對日期均勻於九種有序組合，兩兩獨立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

九種等可能結果，逐一算出碰撞對數

U	V	(B_1, B_2, B_3)	碰撞對數 X
0	0	$(0, 0, 0)$	3
0	1	$(0, 1, 2)$	0
0	2	$(0, 2, 1)$	0
1	0	$(1, 0, 2)$	0
1	1	$(1, 1, 1)$	3
1	2	$(1, 2, 0)$	0
2	0	$(2, 0, 1)$	0
2	1	$(2, 1, 0)$	0
2	2	$(2, 2, 2)$	3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

期望相同，碰撞機率仍可能不同

模型	$\Pr(X=0)$	$\Pr(X=1)$	$\Pr(X=3)$	$\Pr(X>0)$	$\mathbb{E}[X]$
兩兩獨立的構造	$2/3$	0	$1/3$	$1/3$	1
三個日期完全獨立	$2/9$	$2/3$	$1/9$	$7/9$	1

完全獨立時，27 種等可能三元組中，全異有 6 種、全同有 3 種，恰有一對相同則有 $3 \cdot 3 \cdot 2 = 18$ 種：先選哪一對、再選共同日期，最後選另一個不同日期。恰有兩對相同不可能，因為相等具有遞移性。

此模型碰撞率為 $3/9 = 1/3$；三個完全獨立日期的碰撞率則為 $1 - 6/27 = 7/9$。兩者的碰撞對數期望都為 1。相同期望不代表相同分布；精確補事件連乘需要更強的聯合假設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

球入桶：先固定球數，再問負載與空桶數

$t \geq 0$ 顆球各自獨立、均勻投入 $b \geq 1$ 個桶。固定桶的負載 N 是 t 個指示變數的和，故 $\mathbb{E}[N] = t/b$。恰有 r 球入桶須選定成功的 r 次投球，因此

$$\Pr(N = r) = \binom{t}{r} (1/b)^r (1 - 1/b)^{t-r}, \quad 0 \leq r \leq t.$$

令 Z_0、Z_1 分別為空桶數、恰有一球的桶數。對每桶套用指示變數：

$$\mathbb{E}[Z_0] = b(1 - 1/b)^t,$$

$$\mathbb{E}[Z_1] = b \left[t \frac{1}{b} (1 - 1/b)^{t-1} \right] = t(1 - 1/b)^{t-1}, \quad t \geq 1.$$

$t = 0$ 時直接有 $Z_0 = b, Z_1 = 0$；$b = 1$ 的事件直接判定，或以空乘積慣例 $0^0 = 1$ 解讀邊界式。

例 $t = b = 4$：固定桶平均負載為 1，但 $\mathbb{E}[Z_0] = 81/64$、$\mathbb{E}[Z_1] = 27/16$。各桶負載總和固定為四，不獨立；平均一球不代表每桶恰有一球。

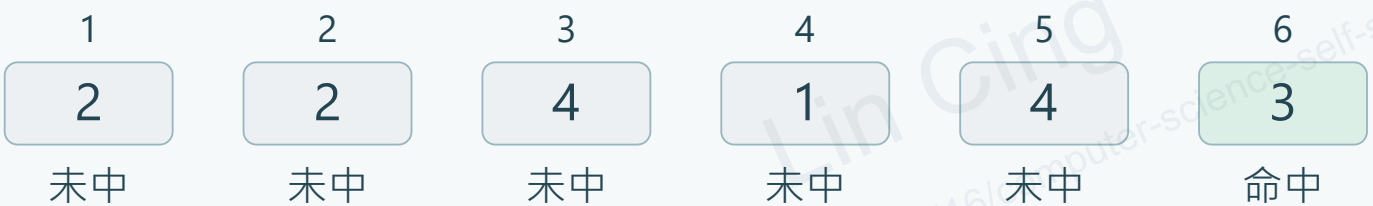
Lin Cing https://github.com/xinhi6716/computer-science-self-study

收集所有桶：每個階段都在等下一個新桶

每次投球獨立、均勻。固定一個桶，令 T 為首次命中所需投球數，含成功的那一次。$\Pr(T > t) = (1 - 1/b)^t$，$t \geq 0$，故尾和給

$$\mathbb{E}[T] = \sum_{t=0}^{\infty} (1 - 1/b)^t = b.$$

四個桶：等到第一次命中指定的第 3 桶



T = 6

E[T] = 4

P(T = 6) = $(3/4)^5 \times (1/4) = 243/4096$

前五次未中，第六次命中；單次等待長度不必等於期望值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每多收集一個新桶，要再等多久

改問全部桶皆命中。已見 i 個不同桶時，新桶機率為 $(b-i)/b$；等待該次新命中的投球數 T_i，條件於目前所見集合，其期望皆為 $b/(b-i)$。未來投球仍獨立且均勻，故不取決於已耗時多久。

總停止時間 $C = \sum_{i=0}^{b-1} T_i$，期望線性性給

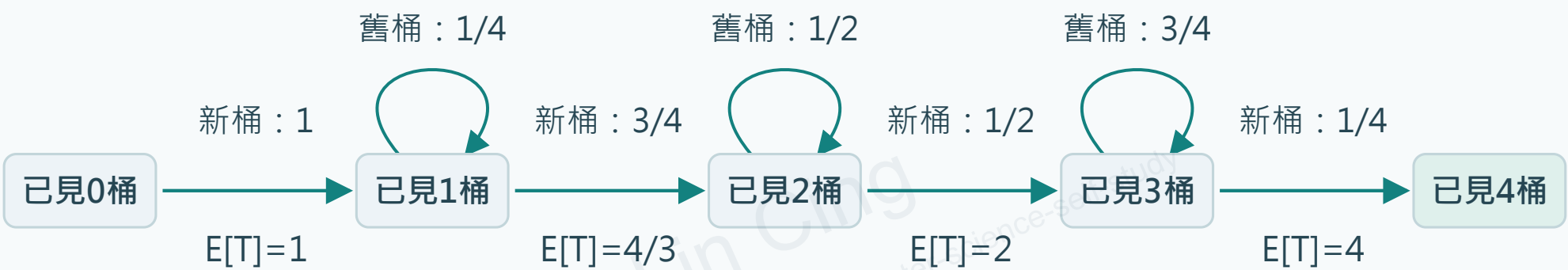
$$\mathbb{E}[C] = \sum_{i=0}^{b-1} \frac{b}{b-i} = bH_b.$$

$b \geq 2$ 時為 $\Theta(b \ln b)$；$b = 1$ 時恰為一次。這個相加步驟不必另證各階段獨立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四桶收集：重複命中時，留在原狀態

命中新桶才前進，命中舊桶留在原狀態



實際投球 2,2,4,1,4,3：第1、3、4、6球命中新桶；各階段長度為1、2、1、2。

階段期望相加為 1+4/3+2+4=25/3；上面的單次投球則在第6球集滿。

四桶各階段期望為 $1, 4/3, 2, 4$，合為 $25/3$。圖中實際投球為 $2, 2, 4, 1, 4, 3$，第 $1, 3, 4, 6$ 球命中新桶，各階段實際長度為 $1, 2, 1, 2$，合為六；單次停止時間不必等於期望。

第一次碰撞與收集完成：停止條件決定時間尺度

先界收集完成時間 C。投完 t 球後，指定桶仍空的機率為 $(1-1/b)^t \le e^{-t/b}$。任一桶空就表示尚未集滿，因此

$$\Pr(C > t) \le \min\{1, be^{-t/b}\}.$$

若整數 $t \ge b(\ln b + c)$、$c \ge 0$，未集滿機率至多 e^{-c}。各桶是否為空可以相依；這裡只用了聯集界。

同一串投球，停止條件不同，答案就不同

t = 1	t = 2	t = 3	t = 4	t = 5	t = 6
2	2	4	1	4	3
{2}	{2}	{2,4}	{1,2,4}	{1,2,4}	{1,2,3,4}

D = 2：首次重複

C = 6：四桶全部出現

每格下方列出截至當次已見的桶；D 與 C 是同一串投球的兩種停止時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

首次碰撞時間：用尾和證明平方根期望

令 D 為第一次出現重複桶的投球編號。$D > k$ 恰表示前 k 球都不同；$k > b$ 時機率為零，其餘回到生日補事件。對 $k \geq 2$，$k(k-1) \geq k^2/2$，故

$$\begin{aligned} \mathbb{E}[D] &= \sum_{k \geq 0} \Pr(D > k) \\ &\leq 2 + \sum_{k \geq 2} e^{-k^2/(4b)} \\ &\leq 2 + \int_0^{\infty} e^{-x^2/(4b)}\,dx = 2 + \sqrt{\pi b}. \end{aligned}$$

下界：對整數 $k \leq \sqrt{b}/2$，聯集界給碰撞率至多 $k(k-1)/(2b) \leq 1/8$，所以前 $\Theta(\sqrt{b})$ 個尾機率至少 $7/8$，得 $\mathbb{E}[D] = \Omega(\sqrt{b})$。

合為 $\mathbb{E}[D] = \Theta(\sqrt{b})$，而 $\mathbb{E}[C] = \Theta(b \ln b)$。同樣是球入桶，問題問「第一次重複」或「全部出現」，答案就不同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

連續正面：視窗數與最長長度是不同的量

獨立、公平擲硬幣 n 次，L 為最長連續正面的長度；沒有正面時 $L=0$。對 $1 \le k \le n$，令 $A_{i,k}$ 表示第 i 到 $i+k-1$ 次全為正面，合法起點為 $1 \le i \le n-k+1$。每個視窗成功率為 2^{-k}，故

$$X_k = \sum_{i=1}^{n-k+1} \mathbf{1}_{A_{i,k}}, \qquad \mathbb{E}[X_k] = (n-k+1)2^{-k}.$$

相鄰視窗可能重疊，不能直接當成獨立事件。例如 $\Pr(A_{1,3} \cap A_{2,3}) = 2^{-4} = 1/16$，因為只需前四次都是正面；兩個邊際機率的乘積卻是 $2^{-6} = 1/64$。期望線性不要求獨立，但求聯集的精確機率需要考慮重疊。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

重疊視窗如何對應同一段連續正面

同一段四連 H 會包含兩個長3視窗，不能當成獨立事件



長3全H視窗：起點1、5、6，共3個。

最長run：5..8，長度4。

重疊視窗可同時成立；第5與第6視窗共享兩次擲幣。

圖中長三的全正面視窗從 $1, 5, 6$ 開始，共 $X_3 = 3$ 個；第 $5..8$ 次的四連正面貢獻了其中兩個視窗。最長長度是 $L = 4$，與視窗數、正面段數都不同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

最長連續正面的上界：先求尾機率，再求期望

事件 $\{L \ge k\}$ 是所有長 k 全正面視窗的聯集。對 $1 \le k \le n$，聯集界給

$$\Pr(L \ge k) \le \min\{1, (n-k+1)2^{-k}\} \le \min\{1, n2^{-k}\}.$$

令 $k_0 = \lceil \lg n \rceil$，其中 $n \ge 1$。由整數尾和及等比級數，

$$\begin{aligned}\mathbb{E}[L] &= \sum_{k=1}^{n} \Pr(L \ge k) \\ &\le k_0 + \sum_{k=k_0+1}^{\infty} n2^{-k} = k_0 + n2^{-k_0} \le \lceil \lg n \rceil + 1.\end{aligned}$$

這裡 $\lg$ 為以 2 為底的對數。若 $k > n$，事件不可能；$n = 0$ 時直接令 $L = 0$，不代入對數。

對 $r \ge 1$ 取整數門檻 $k = \lceil r \lg n \rceil$，可得 $\Pr(L \ge k) \le n^{1-r}$。例如 $n = 1000, k = 20$ 的精細上界為 $981/2^{20} < 10^{-3}$；$k = 30$ 時可用較鬆的 $1000/2^{30} < 10^{-6}$。這些界控制單次執行出現異常長段的機率。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

下界：把硬幣切成互不重疊的獨立區塊

對充分大的 n，令 $s = \lfloor (\lg n)/2 \rfloor \geq 1$、$m = \lfloor n/s \rfloor$。將前 ms 次切成 m 個長 s 的區塊，尾端不足一塊者不用。每塊全正面的機率為 $2^{-s} \geq n^{-1/2}$；不同區塊使用互不重疊的獨立硬幣，因此這些成功事件相互獨立。

若 $L < s$，所有區塊都必須失敗；反方向未必成立，因為長段可能跨越區塊邊界。因此只有下列不等式：

$$\Pr(L < s) \leq (1 - 2^{-s})^{m} \leq e^{-m 2^{-s}} \leq e^{-m/\sqrt{n}} \longrightarrow 0.$$

由 $m = \Theta(n/\lg n)$，有 $m/\sqrt{n} \to \infty$。再利用非負性，

$$\mathbb{E}[L] \geq s \Pr(L \geq s) = \Omega(\lg n).$$

配合上界即得 $\mathbb{E}[L] = \Theta(\lg n)$。這個證明用到了獨立區塊；不能只因重疊視窗的期望很多，就斷言至少一個成功的機率很高。

有限數例：$n = 16, s = 2, m = 8$。至少一個指定區塊為 HH 的機率為 $1 - (3/4)^8 = 58975/65536$，故 $\mathbb{E}[L] \geq 117950/65536$。這是可核對的下界，並非精確期望。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

線上遴選：先固定資訊與決策限制

已知有 $n \geq 2$ 位能力相異的候選人，以均勻隨機排列抵達。每次面試只能與已見者比較；拒絕後不能召回，最多雇用一次。目標為選到全域最佳者，並非讓面試成本最小。

取整數 $1 \leq k < n$，前 k 人只觀察、不雇用，記錄最高分 M；後段遇到第一位超過 M 者便選取。若始終無人超過，則選最後一人。

第二個迴圈不更新 M：在第一個超過門檻者出現以前，其他分數都不高於 M；一旦超過便立即返回。最末面試若未通過比較，仍可直接雇用當前第 n 人，並非召回已離開者。$n = 1$ 直接選唯一者；$k = 0$ 另定義為選第一人。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線上遴選：程序與停止條件

```
ONLINE-SELECT(n,k):
    require n >= 2 and 1 <= k < n
    M = -infinity
    for i = 1 to k:
        M = max(M, score(i))      // interview, then reject
    for i = k+1 to n:
        if score(i) > M: return i
    return n
```

檢查每位後段候選人之前，先前所有後段分數都沒有超過觀察段最高分 M。一旦超過就立即停止；始終沒有超過，才啟用最後一人的備援選擇。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成功機率：條件於最佳者的抵達位置

條件於全域最佳者在位置 i。若 $i \le k$，它已在觀察階段被拒絕，一定失敗。若 $i > k$，成功恰好要求前 $i-1$ 人中的最高分者落在前 k 位：否則後段已有人先超過 M，程序會提早停止；反之，位置 i 就是第一次超過門檻。

最佳者位置均勻，機率為 $1/n$。固定其位置後，前 $i-1$ 人的相對排列仍均勻，因此其中最大者落在前 k 位的條件機率為 $k/(i-1)$。對互斥的位置事件相加，

$$P_k = \sum_{i=k+1}^{n} \frac{1}{n}\frac{k}{i-1} = \frac{k}{n}\sum_{j=k}^{n-1}\frac{1}{j}.$$

推導使用條件機率，沒有直接假設「最佳者位置」與「先前最大者位置」獨立。

邊界也要單獨定義：$n = 1$ 時成功率 1；$k = 0$ 時選第一人，成功率 $1/n$。式中的調和和從 $k \ge 1$ 起才有定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最佳門檻：有限差分與漸近極限分開求

令 $q_k = \sum_{j=k}^{n-1} 1/j$，並定義空和 $q_n = 0$。因 $q_k = 1/k + q_{k+1}$，對 $1 \le k \le n-2$ 有

$$P_{k+1} - P_k = \frac{(k+1)q_{k+1} - k q_k}{n} = \frac{q_{k+1} - 1}{n}.$$

q_k 隨 k 嚴格下降，故成功率先增後減。門檻 k 滿足 $q_k \ge 1 \ge q_{k+1}$ 時達最大；等號可能使相鄰門檻並列。$n = 2$ 的唯一正門檻為 $k = 1$，與 $k = 0$ 同為 $1/2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最佳成功率為何趨近 1/e

遞減函數 $1/x$ 的積分夾界給

$$\ln(n/k) \le q_k \le \ln(n/k) + 1/k,$$

因此 $f(k/n) \le P_k \le f(k/n) + 1/n$，其中 $f(x) = x\ln(1/x)$。由 $f'(x) = \ln(1/x) - 1$、$f''(x) = -1/x < 0$，最大值在 $x = 1/e$，等於 $1/e$。

選離 n/e 最近的合法整數，使 k/n 距 $1/e$ 為 $O(1/n)$；連續可微性給 $f(k/n) = 1/e - O(1/n)$。所有門檻又有 $P_k \le 1/e + 1/n$，故最佳成功率趨近 $1/e$。有限 n 的最佳整數門檻仍應用差分求，不能直接把 n/e 當成精確答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

六人遴選：門檻機率與每次決策都能核對

取 $n = 6$，將各門檻代入 $P_k = (k/6)q_k$：

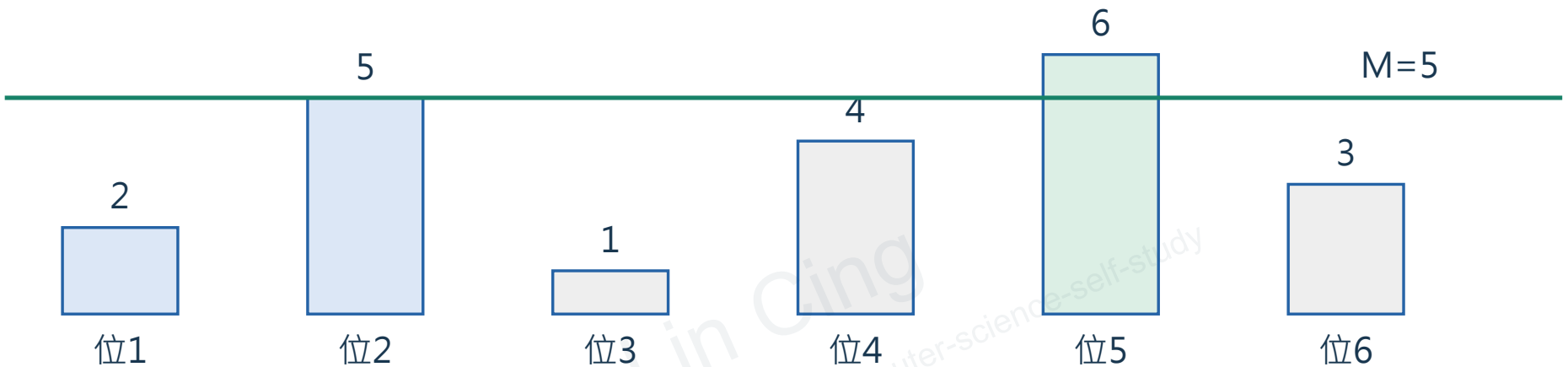
k	q_k	精確成功率 P_k
1	$137/60$	$137/360$
2	$77/60$	$77/180$
3	$47/60$	$47/120$
4	$9/20$	$3/10$
5	$1/5$	$1/6$

$q_2 > 1 > q_3$，故最佳正門檻是 $k = 2$；$k = 0$ 成功率只有 $1/6$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從觀察門檻到每一位候選人的決策

排名 [2,5,1,4,6,3]：取樣最高5，第5人首次超越門檻



排名 $[2, 5, 1, 4, 6, 3]$：前兩人拒絕，$M = 5$；第三人 $1 \leq 5$、第四人 $4 \leq 5$，均拒絕；第五人 $6 > 5$，立即雇用，第六人無須面試。失敗例 $[5, 6, 1, 2, 3, 4]$ 則在取樣時拒絕了最佳者，後段無人超過 6，最後按規則選 4。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

隨機遞迴分割：條件期望與葉數守恆

定義一個額外的成本模型：大小 $n \ge 2$ 的呼叫先輸出 n 個星號，再均勻選 $p \in \{1, \dots, n-1\}$，處理大小 $p, n-p$ 的兩個子問題；大小 1 輸出一個驚嘆號後停止。每次抽樣在給定先前歷史後仍均勻。

令 L_n, S_n 分別為驚嘆號、星號的期望，$L_1 = 1, S_1 = 0$。固定分割點 p 後，兩側期望相加；再平均所有 p，利用 $j \leftrightarrow n-j$ 的對稱，

$$L_n = \frac{2}{n-1} \sum_{j=1}^{n-1} L_j, \qquad S_n = n + \frac{2}{n-1} \sum_{j=1}^{n-1} S_j.$$

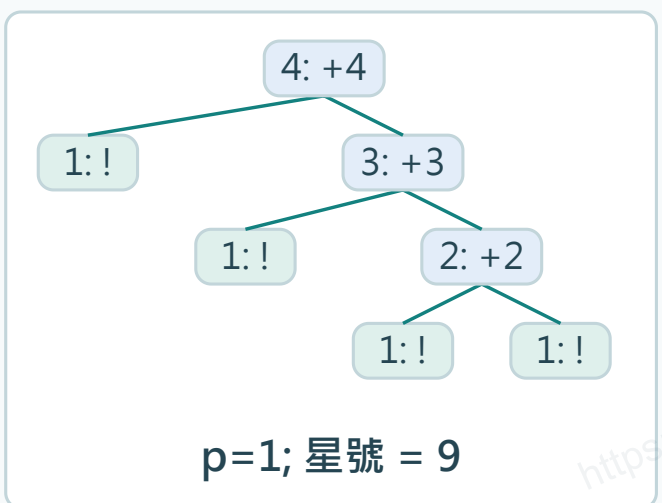
不需要假設兩側成本獨立，因為用的是期望的加法。每條分支的整數規模都嚴格下降，故必然終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

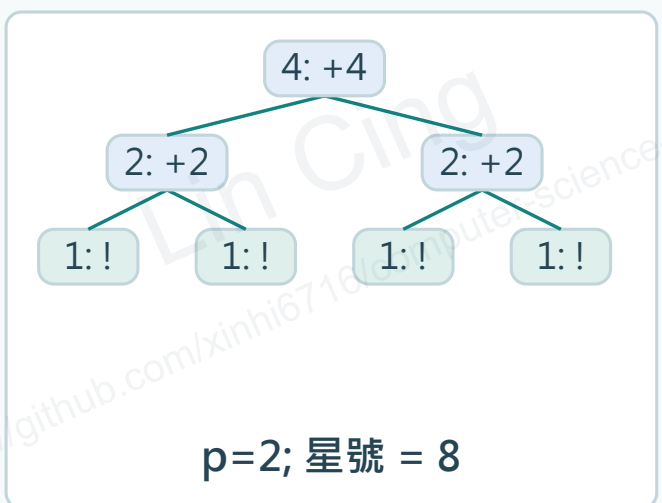
四筆分割：葉數相同，成本卻不同

每次分割保存大小總和，最終每片葉的大小都是 1，所以**每次執行都恰有 n 片葉**，不只是期望如此。星號數卻會改變：$n = 4$ 時根成本為 4；平衡分成 $2 + 2$，總成本 $4 + 2 + 2 = 8$；分成 $1 + 3$ 時，大小 3 的成本為 $3 + 2 = 5$，總成本為 9。因此期望 $S_4 = (9 + 8 + 9)/3 = 26/3$。

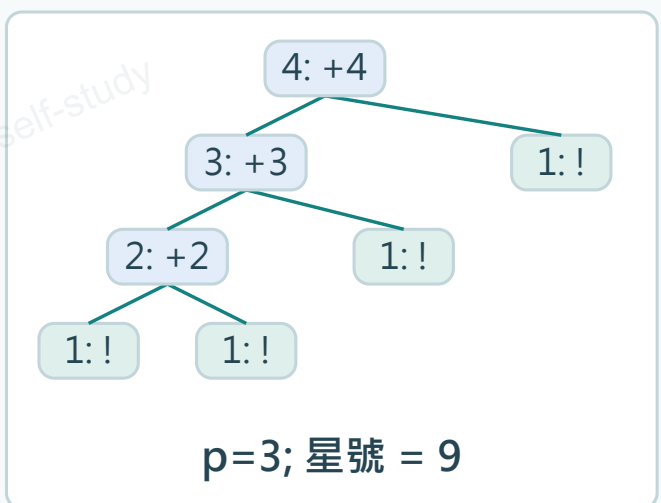
三種根分割各機率 1/3；每棵樹都有四片葉



p=1; 星號 = 9



p=2; 星號 = 8



p=3; 星號 = 9

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

解出隨機分割成本：相鄰規模相減

葉數可用強歸納核對：若 $j<n$ 時 $L_j=j$，則 $L_n=\frac{2}{n-1}\sum_{j<n} j=n$。

星號成本先乘開分母：

$$(n-1)S_n = n(n-1) + 2\sum_{j<n} S_j,$$

$$nS_{n+1} = n(n+1) + 2\sum_{j<n} S_j + 2S_n.$$

兩式相減得 $nS_{n+1}-(n-1)S_n = 2n+2S_n$，所以 $S_{n+1}=2+\frac{n+1}{n}S_n$。此差分對 $n\ge 2$ 由上式成立，$n=1$ 可由 $S_1=0, S_2=2$ 直接核對。

令 $Q_n=S_n/n$，則 $Q_{n+1}=Q_n+2/(n+1)$。由 $Q_1=0$ 累加，得

$$Q_n = 2(H_n-1), \qquad S_n = 2n(H_n-1) = \Theta(n\ln n).$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

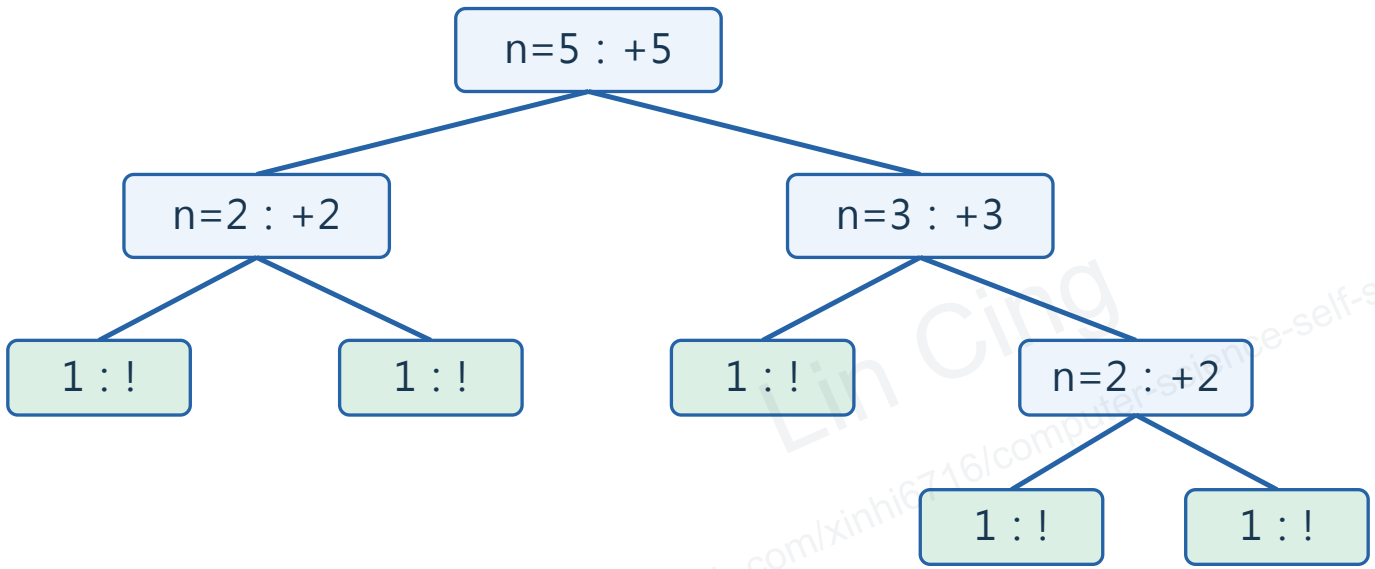
用小規模輸入核對封閉式

n	每次執行葉數	$\mathbb{E}[\text{星號數}]$
2	2	2
3	3	5
4	4	$26/3$
5	5	$77/6$
6	6	$87/5$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

期望成本與最壞執行仍須分開

五個單位葉完整展開：守恆的是葉數，不是星號成本



本次葉數=5

本次星號=12

E[星號]=77/6

+數字：本輪星號

!：一個驚嘆號

圖示 p=2、之後 3=1+2；葉數始終5，但星號總數會隨分割而變。

最壞路徑每次切出 1，星號數為 $2+3+\cdots+n=n(n+1)/2-1$；期望的線性對數界並不是每次執行的保證。

參考資料與署名

參考用書：Thomas H. Cormen、Charles E. Leiserson、Ronald L. Rivest、Clifford Stein，Introduction to Algorithms，第 3 版，第 5–9 章。 補充講義：MIT 6.046J/18.401J，Fall 2005。Lecture 4: Quicksort（Charles E. Leiserson，2005-09-21）；Lecture 5: Sorting Lower Bounds / Linear-Time Sorting（Erik Demaine，2005-09-26）；Lecture 6: Order Statistics（Erik Demaine，2005-09-28）。

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study